

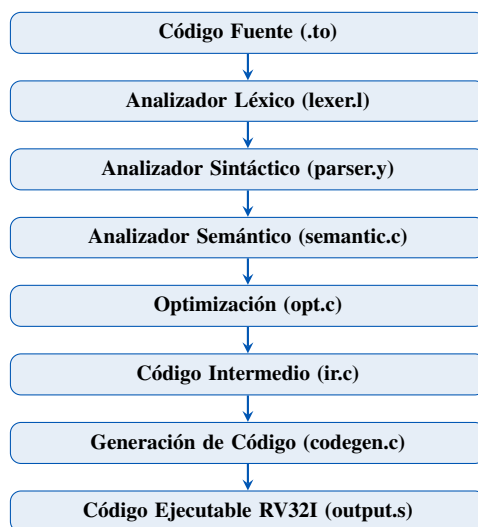


**UNIVERSIDAD NACIONAL
JORGE BASADRE GROHMANN**

Facultad de Ingeniería
Escuela Profesional de Ingeniería en Informática y Sistemas

COMPILADORES Y TEORÍA DE LENGUAJES INFORME

Compilador final de un minilenguaje



Docente:

Dr. Apaza Valencia Manuel Yuri

Autor(es):

Cachicatari Quito Aron Rene,  [0009-0001-0343-6817](#)

Chata Chino Alex Jeanpier,  [0009-0006-0901-569X](#)

Condori Caljaro Luis Fernando,  [0009-0000-9748-1557](#)

Mamani Catacora Johan Anibal,  [0009-0006-3486-7579](#)

Flores Gómez Mauricio José,  [0000-0002-1840-3161](#)

Agradecemos a nuestro estimado Ing. Manuel Apaza Valencia, que con su paciencia y exigencia nos llevó a realizar este simple, pero interesante compilador. Asimismo, al Ing. Acero, por sus enseñanzas para el entendimiento y el enlace con el simulador Logisim Evolution.

A todos, este proyecto es para ustedes: un compilador no es difícil.

— Los autores

Índice general

INTRODUCCIÓN	10
1. ANTECEDENTES DE LA INVESTIGACIÓN	11
1.1. Antecedentes	11
1.2. Definición del Lenguaje TPP	11
1.2.1. Objetivos del Lenguaje	11
1.2.2. Características Sintácticas	12
1.2.3. Restricciones del Lenguaje	12
1.3. Especificación Léxica	13
1.3.1. Palabras Reservadas	13
1.3.2. Clases de Tokens	13
1.4. Expresiones Regulares y Autómatas Finitos	13
1.5. Diseño de la Gramática (BNF)	14
1.5.1. Eliminación de Ambigüedad	14
1.6. Rúbrica de Evaluación	15
1.7. Objetivos	16
1.7.1. Objetivo General	16
1.7.2. Objetivos Específicos	16
2. Introducción al Compilador TPP	17
2.1. Descripción General	17
2.2. Arquitectura General del Compilador	17
2.3. Características del Lenguaje TPP	18
2.4. Arquitectura Objetivo: RISC-V RV32I	19
2.5. Estructura de Archivos del Proyecto	19
3. FRONTEND — Análisis Léxico	20
3.1. Fundamentos Teóricos	20
3.2. Implementación con Flex: <code>lexer.l</code>	21
3.2.1. Configuración y Expresiones Regulares Base	21
3.2.2. Reglas de Producción Léxica	22

3.2.3. Tabla Completa de Tokens	23
3.3. Detección y Manejo de Errores Léxicos	23
4. FRONTEND — Análisis Sintáctico	24
4.1. Fundamentos Teóricos	24
4.2. Implementación con Bison: <code>parser.y</code>	25
4.2.1. Funcionamiento Interno (Shift/Reduce)	25
4.2.2. Gramática del Lenguaje TPP	25
4.2.3. Resolución de Ambigüedades	27
4.3. Estructuras Sintácticas Soportadas	27
4.3.1. Declaraciones de Variables	27
4.3.2. Declaraciones de Funciones	27
4.3.3. Estructuras de Control	28
4.4. Manejo de Errores Sintácticos	28
4.5. Orquestación del Pipeline	29
5. FRONTEND — Análisis Semántico y Tabla de Símbolos	30
5.1. Fundamentos Teóricos	30
5.2. Tabla de Símbolos: <code>syntab.c</code>	30
5.2.1. Estructura del Símbolo	31
5.2.2. Gestión de Memoria	31
5.2.3. Scoping y Eliminación de Ámbitos	32
5.3. Implementación: <code>semantic.c</code>	32
5.3.1. Anotación de Nodos del AST	32
5.3.2. Verificación de Tipos	33
5.4. Errores Semánticos Detectados	34
5.5. Visualización de la Tabla de Símbolos	34
6. BACKEND — El Árbol Sintáctico Abstracto (AST)	36
6.1. Fundamentos Teóricos: AST vs Parse Tree	36
6.2. Estructura del Nodo	37
6.3. Tipos de Nodo	39
6.4. Ejemplo de AST	39
6.5. Construcción de Nodos	40
7. BACKEND — Optimización del AST	42
7.1. Fundamentos Teóricos	42
7.1.1. Justificación del Orden de Ejecución	42
7.2. Optimización 1: Plegado de Constantes	43
7.2.1. Descripción	43

7.2.2.	Operaciones soportadas	44
7.2.3.	Implementación	44
7.2.4.	Ejemplo	45
7.3.	Optimización 2: Simplificación Algebraica	45
7.3.1.	Descripción	45
7.4.	Optimización 3: Eliminación de Código Muerto	46
7.4.1.	Descripción	46
7.4.2.	Casos detectados	46
7.5.	Optimización 4: Propagación de Constantes	46
7.5.1.	Descripción	47
7.6.	Informe de Optimizaciones	47
7.7.	Posibilidades de Optimización en el Código Final	47
8.	BACKEND — Código Intermedio (TAC)	48
8.1.	Fundamentos Teóricos	48
8.2.	Implementación: <code>ir.c</code>	49
8.2.1.	Formato TAC	49
8.2.2.	Generador Recursivo	49
8.3.	Ejemplo de TAC	51
8.4.	TAC para Estructuras de Control	51
8.4.1.	Condicional si/sino	51
8.4.2.	Bucle para	52
9.	BACKEND — Generación de Código RV32I	53
9.1.	Fundamentos Teóricos: Arquitectura RISC-V RV32I	53
9.1.1.	Registros Arquitectónicos RV32I	53
9.1.2.	Convención de Llamadas (Calling Convention)	53
9.2.	Modelo de Memoria	54
9.2.1.	Stack Único Compartido	54
9.3.	Generación Bifásica para Funciones	55
9.4.	Rutinas de Soporte Emitidas Automáticamente	56
9.4.1.	Multiplicador por software	56
9.5.	Generación por Tipo de Nodo	56
9.5.1.	Expresiones Aritméticas	56
9.5.2.	Estructuras de Control	58
9.5.3.	Funciones: Prólogo, Llamada y Epílogo	58
9.5.4.	Instrucción imprimir	59
9.6.	Tabla de Posibilidades del Código Generado	60
9.7.	Salida de Diagnóstico	62

10. METODOLOGÍA — Integración con Logisim Evolution	64
10.1. Visión General del Proceso	64
10.2. El Script <code>compile_to_logisim.py</code>	65
10.2.1. Origen y Adaptación	65
10.2.2. Funcionamiento Interno	65
10.2.3. Codificación de Ejemplo	66
10.3. Arquitectura RISC-V y el Procesador Logisim	66
10.3.1. El Procesador de Ciclo Único	66
10.3.2. Ciclo de Ejecución por Instrucción	67
11. COMPLICACIONES	68
11.1. Panorama General de Dificultades	68
11.2. Complicación 1: Ausencia de Instrucción <code>mul</code> en RV32I Base	68
11.2.1. Descripción	68
11.2.2. Solución Adoptada	68
11.3. Complicación 2: Salida TTY en Logisim	69
11.3.1. Descripción	69
11.3.2. Problema de Codificación	69
11.3.3. Solución	69
11.4. Complicación 3: Modelo de Memoria Estático y Funciones Recursivas	69
11.4.1. Descripción	70
11.4.2. Manifestación del Error	70
11.4.3. Causa Técnica	70
11.4.4. Solución Actual	70
11.5. Complicación 4: Formato del Archivo Hexadecimal para Logisim	70
11.5.1. Descripción	70
11.5.2. Problema Inicial	71
11.5.3. Solución	71
11.6. Complicación 5: Conflictos Léxicos con Babel en LaTeX	71
11.6.1. Descripción	71
11.6.2. Solución	71
11.7. Complicación 6: Caracteres Especiales en Cadenas de Texto	71
11.7.1. Descripción	71
11.7.2. Solución	72
12. PRUEBAS Y DEMOSTRACIONES DEL COMPILADOR	73
12.1. Descripción de las Pruebas	73
12.2. Prueba 1: Programa Correcto — Operaciones Básicas	73
12.2.1. Código Fuente	73
12.2.2. Salida Esperada del Compilador	73

12.3. Prueba 2: Error Léxico	76
12.3.1. Código Fuente con Error	76
12.3.2. Salida del Compilador	76
12.4. Prueba 3: Error Sintáctico	77
12.4.1. Código Fuente con Error	77
12.4.2. Salida del Compilador	77
12.5. Prueba 4: Error Semántico	78
12.5.1. Código Fuente con Error	78
12.5.2. Salida del Compilador	78
12.6. Prueba 5: Programa con Funciones	79
12.6.1. Código Fuente	79
12.6.2. Salida Esperada	79
12.7. Prueba 6: Optimización de Constantes	79
12.7.1. Código Fuente	80
12.7.2. Salida del Optimizador	80
12.8. Prueba 7: Programa con Bucles y Arreglos	81
12.8.1. Código Fuente	81
12.8.2. Salida Esperada	81
12.9. Prueba 8: Prueba Integral — Backend	81
12.9.1. Código Fuente	81
12.9.2. Resultado en Logisim	82
13. MANUAL DE USUARIO	84
13.1. Sintaxis del Lenguaje TPP	84
13.1.1. Tipos de Datos y Variables	84
13.1.2. Entrada y Salida	84
13.1.3. Expresiones y Operadores	85
13.1.4. Estructuras Condicionales: si / sino	85
13.1.5. Selección Múltiple: depende / caso	85
13.1.6. Bucles: mientras y para	86
13.1.7. Funciones: fun	87
13.2. Guía de Compilación y Ejecución en Logisim	87
13.2.1. 1. Compilación del código fuente	87
13.2.2. 2. Generación del Código Hexadecimal	88
13.2.3. 3. Carga en Logisim Evolution	88
14. Conclusiones y Trabajo Futuro	89
14.1. Resumen del Proceso de Compilación	89
14.2. Logros Implementados	89
14.2.1. Frontend	89

14.2.2. Backend	90
14.3. Construcciones Soportadas	91
14.4. Limitaciones Actuales	91
14.4.1. Modelo de Memoria Estático	91
14.4.2. Sin guión bajo en identificadores	92
14.5. Trabajo Futuro	92
14.6. Ejemplo Completo: De Fuente a Binario	92

Índice de figuras

2.1. Pipeline completo del compilador TPP	18
3.1. Autómata Finito Determinista (DFA) del Lexer de TPP. Los estados con doble borde (verde) son estados de aceptación . Rama superior (azul): $q_0 \rightarrow q_1$, identificador o palabra reservada. Rama media: $q_0 \rightarrow q_2$, entero; $q_2 \rightarrow q_3 \rightarrow q_4$, decimal. Rama inferior (rojo): $q_0 \rightarrow q_5 \rightarrow q_6 \rightarrow q_7$, comentario de línea.	21
4.1. Ejemplo de Árbol de Derivación (Parse Tree) para la expresión matemática $3 + x * 5$, resolviendo la precedencia mediante la gramática.	24
6.1. Comparación entre el Parse Tree clásico (izquierda) y el AST generado por TPP (derecha) para la sentencia entero $x = 3 + 5$; . El AST descarta nodos superfluos (;, =, tipo entero, nodos intermedios) y conserva únicamente la estructura lógica: asignación, variable y expresión sumada.	37
6.2. AST generado por TPP para <code>fun main(){ entero resultado = 10*2; imprimir(resultado); }</code> . Nodos azules = nodos internos con semántica; nodos verdes = hojas terminales (literales o identificadores).	37
7.1. Visualización del Plegado de Constantes. El subárbol de la operación constante $(3 + 5)$ colapsa en un solo nodo hoja constante (8).	45
7.2. Visualización de la Simplificación Algebraica. La operación de suma con el elemento neutro 0 ($x + 0$) se elimina, promoviendo el nodo de la variable (x) para reemplazar a su padre.	46
9.1. Esquema de traducción post-orden. El generador primero emite el código para los hijos (izquierdo y derecho) almacenando sus resultados en registros temporales, y finalmente emite la instrucción que los combina.	60
10.1. Pipeline completo hasta ejecución en Logisim	64
12.1. Ejecución del programa mostrando el árbol sintáctico	74
12.2. Ejecución del programa mostrando análisis semántico y tabla de símbolos	74

12.3. Ejecución del programa mostrando el código intermedio (TAC)	75
12.4. Ejecución del programa mostrando las optimizaciones aplicadas	75
12.5. Ejecución del programa mostrando el código final generado	76
12.6. Captura de error léxico por carácter inválido	77
12.7. Captura de error sintáctico por paréntesis faltante	77
12.8. Captura de error semántico por variable no declarada	78
12.9. Ejecución del programa con funciones	79
12.10Ejecución mostrando optimizaciones aplicadas	80
12.11Ejecución del programa con arreglos y bucles para	81
12.12Ejecución del programa en Logisim Evolution — salida TTY	83

Índice de cuadros

1.1. Sintaxis del lenguaje TPP	12
1.2. Especificación léxica del lenguaje TPP	13
1.3. Rúbrica de evaluación del compilador TPP	15
2.1. Características del lenguaje TPP	18
2.2. Archivos del compilador TPP	19
3.2. Mapeo de expresiones regulares a Tokens en TPP	23
5.1. Errores semánticos detectados por el compilador	34
6.2. Tipos de nodos del AST	39
7.1. Operaciones sujetas a plegado de constantes	44
7.2. Identidades algebraicas aplicadas	45
7.3. Impacto de las optimizaciones en el código generado	47
9.1. Registros utilizados por el compilador TPP en RV32I	53
9.2. Rutinas de soporte generadas automáticamente	56
9.3. Mapeo de construcciones TPP a instrucciones RV32I	60
10.1. Codificación de <code>addi sp, sp, -32</code> en RV32I formato I	66
14.1. Resumen del pipeline completo	89
14.2. Estado de las características del compilador	91

INTRODUCCIÓN

Un compilador es un sistema de software encargado de procesar un programa escrito en un lenguaje fuente y transformarlo en una representación válida para su ejecución, interpretación o traducción posterior. Este proceso no se limita al reconocimiento de palabras y estructuras gramaticales, sino que integra varias fases relacionadas entre sí: análisis léxico, análisis sintáctico, análisis semántico, generación de código intermedio y optimización.

En el avance previo del proyecto se presentó la base inicial del compilador: el analizador léxico desarrollado con Flex y el analizador sintáctico construido con Bison para un minilenguaje estructurado en español. Esta primera etapa permitió reconocer tokens, validar la gramática del lenguaje y detectar errores sintácticos durante el procesamiento del código fuente.

En la presente versión final, el proyecto se amplía hasta consolidar un compilador más completo. Para ello, se incorporó el análisis semántico, encargado de verificar la coherencia del programa más allá de su forma gramatical; la generación de código intermedio, que permite representar las instrucciones en una forma más cercana al procesamiento interno del compilador; y la optimización de código, orientada a simplificar o mejorar dicha representación antes de obtener el resultado final.

Asimismo, el uso de un minilenguaje con palabras reservadas en español mantiene un enfoque didáctico, ya que facilita la comprensión de las estructuras básicas de programación y del funcionamiento interno de un compilador. De esta manera, el proyecto permite observar de forma progresiva cómo un código fuente pasa por distintas etapas hasta convertirse en una representación validada, organizada y optimizada.

Finalmente, este informe describe la evolución del compilador desde sus fases iniciales hasta su versión terminada, resaltando la importancia de integrar análisis léxico, análisis sintáctico, análisis semántico, código intermedio y optimización dentro de una arquitectura funcional de compilación.

Capítulo 1 ANTECEDENTES DE LA INVESTIGACIÓN

1.1 Antecedentes

El desarrollo de compiladores ha evolucionado desde procesos manuales de traducción hasta sistemas automatizados capaces de analizar, optimizar y generar código de manera eficiente. Dentro de este proceso, las fases de análisis léxico y sintáctico cumplen un papel esencial como punto de partida, pero un compilador completo requiere además análisis semántico, generación de código intermedio y optimización para garantizar que el programa no solo sea gramaticalmente correcto, sino también coherente y procesable.

Herramientas como Lex/Flex y Yacc/Bison han sido ampliamente utilizadas en el ámbito académico y profesional para la construcción de analizadores. Flex facilita la definición de patrones mediante expresiones regulares, mientras que Bison permite especificar gramáticas y reglas de producción. Sobre esta base, las fases posteriores del compilador permiten validar reglas semánticas, construir representaciones intermedias y aplicar optimizaciones antes de producir el resultado final.

1.2 Definición del Lenguaje TPP

1.2.1 Objetivos del Lenguaje

El lenguaje **TPP** (*Tiny Programming Processor*) fue diseñado con los siguientes objetivos pedagógicos:

- Permitir que los estudiantes de sistemas escriban programas imperativos en un lenguaje cuya sintaxis está en **español**, reduciendo la carga cognitiva del idioma extranjero al aprender teoría de compilación.
- Ser lo suficientemente completo para ilustrar todas las fases del pipeline de compilación (léxico, sintáctico, semántico, código intermedio, optimización y generación de código).

- Generar código ejecutable en hardware real (simulado en **Logisim Evolution**) a través de la ISA RISC-V RV32I.

1.2.2 Características Sintácticas

La sintaxis de TPP sigue un estilo imperativo similar a C/Java pero con palabras reservadas en español. A continuación se presenta el resumen de sus construcciones:

Categoría	Construcción	Ejemplo en TPP
Tipos de datos	Entero, Decimal	<code>entero x = 5;</code>
Arreglos	Estáticos de tamaño fijo	<code>entero arr[10];</code>
Control de flujo	si/sino	<code>si (x > 0) { ... } sino { ... }</code>
Bucles	mientras, para	<code>mientras (i < 10) { i = i + 1; }</code>
Selección múltiple	depende/caso/otros	<code>depende (x) { caso 1: { ... } }</code>
Funciones	Con parámetros y retorno	<code>fun suma(entero a, entero b) -> entero { ... }</code>
Entrada/Salida	imprimir, leer	<code>imprimir("Hola mundo");</code>
Comentarios	Línea simple	<code>// Este es un comentario</code>
Operadores arit.	+, -, *, /, %	<code>x = a * 2 + b / 3;</code>
Operadores relac.	==, !=, <, >, <=, >=	<code>si (a != b) { ... }</code>
Retorno de función	retornar	<code>retornar a + b;</code>

Cuadro 1.1: Sintaxis del lenguaje TPP

1.2.3 Restricciones del Lenguaje

- Los identificadores solo pueden contener letras y dígitos (sin guión bajo `_`). Se recomienda *camelCase*.
- No existe el tipo booleano explícito; las comparaciones devuelven 0 (falso) o 1 (verdadero).
- Los arreglos deben declararse con tamaño constante conocido en tiempo de compilación.
- No hay recursión completa (modelo de memoria estático sin frames dinámicos).

1.3 Especificación Léxica

1.3.1 Palabras Reservadas

Las palabras reservadas de TPP son las siguientes y no pueden usarse como identificadores:

fun si sino mientras para retornar imprimir leer entero
decimal depende caso otros

1.3.2 Clases de Tokens

Clase	Expresión Regular	Token	Ejemplo
Entero	$[0-9]^+$	NUM_ENTERO	42
Decimal	$[0-9]^+ \backslash . [0-9]^+$	NUM_DECIMAL	3.14
Cadena	$"([\^"\\] \\ \.)^*"$	CADENA	"hola"
Identificador	$[a-zA-Z][a-zA-Z0-9]^*$	ID	miVar
Suma	+	MAS	+
Resta	-	MENOS	-
Multiplicación	*	MULT	*
División	/	DIV	/
Igual	==	IGUAL	==
Diferente	!=	DIFERENTE	!=
Flecha retorno	->	FLECHA	->

Cuadro 1.2: Especificación léxica del lenguaje TPP

1.4 Expresiones Regulares y Autómatas Finitos

Las categorías léxicas de TPP se definen formalmente mediante **Expresiones Regulares (ER)**. Flex convierte internamente estas ER en un **Autómata Finito No Determinista (NFA)** utilizando la construcción de Thompson, y luego aplica la construcción de subconjuntos para obtener un **Autómata Finito Determinista (DFA)** equivalente optimizado.

Por ejemplo, la ER para un número entero es:

$$\text{DIGITO}^+ \equiv [0-9][0-9]^*$$

Y la ER para un identificador es:

$$\text{LETRA} \cdot (\text{LETRA} \mid \text{DIGITO})^* \equiv [a-zA-Z][a-zA-Z0-9]^*$$

El DFA generado puede procesar cadenas de entrada en tiempo $O(n)$ proporcional a la longitud de la entrada, sin retroceder nunca (propiedad de los DFA).

1.5 Diseño de la Gramática (BNF)

La gramática del lenguaje TPP es una **Gramática Libre de Contexto (CFG)** escrita en formato BNF. A continuación se presentan las reglas más representativas:

```

1 programa      ::= lista_elementos
2 lista_elementos ::= elemento | lista_elementos elemento
3
4 elemento       ::= declaracion_fun
5                  | declaracion_var
6                  | instruccion
7
8 declaracion_fun ::= "fun" ID "(" parametros ")" "->" tipo "{"
9                  bloque "}"
10                  | "fun" ID "(" parametros ")" "{" bloque "}"
11
12 declaracion_var ::= tipo lista_ids ";"
13                  | tipo ID "[" NUM_ENTERO "]" ";"
14
15 instruccion     ::= estructura_si
16                  | estructura_mientras
17                  | estructura_para
18                  | estructura_depends
19                  | asignacion ";"
20                  | llamada_funcion ";"
21                  | "retornar" expresion ";"
22                  | "imprimir" "(" lista_impresion ")" ";"
23                  | "leer" "(" ID ")" ";"
24
25 expresion       ::= expresion "+" expresion
26                  | expresion "-" expresion
27                  | expresion "*" expresion
28                  | expresion "/" expresion
29                  | expresion "==" expresion
30                  | expresion "!=" expresion
31                  | expresion "<" expresion
32                  | expresion ">" expresion
33                  | NUM_ENTERO | NUM_DECIMAL | ID | CADENA
34                  | llamada_funcion

```

Listing 1.1: Extracto de la gramática en BNF — parser.y

1.5.1 Eliminación de Ambigüedad

La gramática de expresiones es inherentemente ambigua (no especifica precedencia). Bison resuelve esto con declaraciones de precedencia explícitas en el archivo `parser.y`:


```

1 %left IGUAL DIFERENTE MENOR MAYOR MENOR_IGUAL MAYOR_IGUAL //
  menor prec.
2 %left MAS MENOS
3 %left MULT DIV MOD //
  mayor prec.
4
5 // Resolver el "dangling else":
6 %nonassoc LOWER_THAN_SINO
7 %nonassoc SINO

```

Listing 1.2: Declaraciones de precedencia en parser.y

Esto garantiza que $3 + 4 * 2$ se evalúe como $3 + (4*2) = 11$ y que un sino siempre se asocie con el si más cercano.

1.6 Rúbrica de Evaluación

El proyecto fue evaluado bajo la siguiente rúbrica:

Nº	Criterio	Excelente	Bueno	Regular	Puntaje
1	Definición del lenguaje (objetivos, sintaxis, características)	1.0	0.75	0.50	1.0
2	Especificación léxica (tokens, reservadas, operadores)	1.5	1.0	0.5	1.5
3	Expresiones regulares y autómatas finitos	1.5	1.0	0.5	1.5
4	Implementación del analizador léxico	2.0	1.5	1.0	2.0
5	Diseño de la gramática (BNF/EBNF) y eliminación de ambigüedad	2.0	1.5	1.0	2.0
6	Implementación del analizador sintáctico	2.0	1.5	1.0	2.0
7	Implementación del analizador semántico y tabla de símbolos	2.0	1.5	1.0	2.0
8	Generación de código intermedio	1.5	1.0	0.5	1.5
9	Optimización de código	1.0	0.75	0.50	1.0
10	Generación de código final (ensamblador RV32I)	2.0	1.5	1.0	2.0
11	Funcionamiento integral (pruebas y manejo de errores)	1.5	1.0	0.5	1.5
12	Documentación técnica, presentación y defensa del proyecto	2.0	1.5	1.0	2.0
TOTAL					20.0

Cuadro 1.3: Rúbrica de evaluación del compilador TPP

1.7 Objetivos

1.7.1 Objetivo General

Diseñar e implementar la versión final de un compilador para un minilenguaje estructurado en español, integrando análisis léxico, análisis sintáctico, análisis semántico, generación de código intermedio y optimización de código, con salida en ensamblador RISC-V RV32I ejecutable en Logisim Evolution.

1.7.2 Objetivos Específicos

- **Consolidar las fases iniciales:** Mantener e integrar el analizador léxico en Flex y el analizador sintáctico en Bison como base del compilador del minilenguaje.
- **Implementar el análisis semántico:** Verificar reglas de coherencia del programa, como uso de identificadores, compatibilidad de tipos y validez de instrucciones.
- **Generar código intermedio:** Transformar las instrucciones reconocidas en una representación intermedia (TAC) que facilite su posterior procesamiento.
- **Optimizar el código generado:** Aplicar plegado de constantes, simplificación algebraica y eliminación de código muerto.
- **Generar código RV32I:** Traducir el AST optimizado a instrucciones ensamblador RISC-V ejecutables en el procesador Logisim.
- **Integrar con Logisim:** Utilizar `compile_to_logisim.py` para convertir el ensamblador a formato hexadecimal cargable directamente en la RAM del procesador.
- **Validar el compilador final:** Probar el sistema con programas del minilenguaje que evidencien el funcionamiento integrado de todas las fases.

Capítulo 2 Introducción al Compilador TPP

2.1 Descripción General

El compilador **TPP** (*Tiny Programming Processor*) es un compilador completo de un lenguaje de programación imperativo de propósito educativo, diseñado con sintaxis en español. El objetivo fundamental del proyecto es traducir código fuente escrito en el lenguaje TPP a instrucciones de ensamblador **RISC-V RV32I**, que luego pueden ser ejecutadas directamente en un procesador RISC-V de ciclo único implementado en **Logisim Evolution**.

Objetivo del Compilador TPP

Transformar código fuente en español al lenguaje ensamblador RV32I, permitiendo su ejecución en hardware real simulado dentro de Logisim. El resultado final (`output.s`) puede cargarse directamente en la RAM del circuito para su ejecución.

2.2 Arquitectura General del Compilador

El compilador sigue el pipeline clásico de traducción dividido en dos grandes etapas:

- **Frontend:** Análisis del código fuente para verificar su validez léxica, sintáctica y semántica. Produce el Árbol Sintáctico Abstracto (AST).
- **Backend:** Transformación del AST en código ejecutable, pasando por optimización, representación intermedia y generación final de ensamblador RV32I.

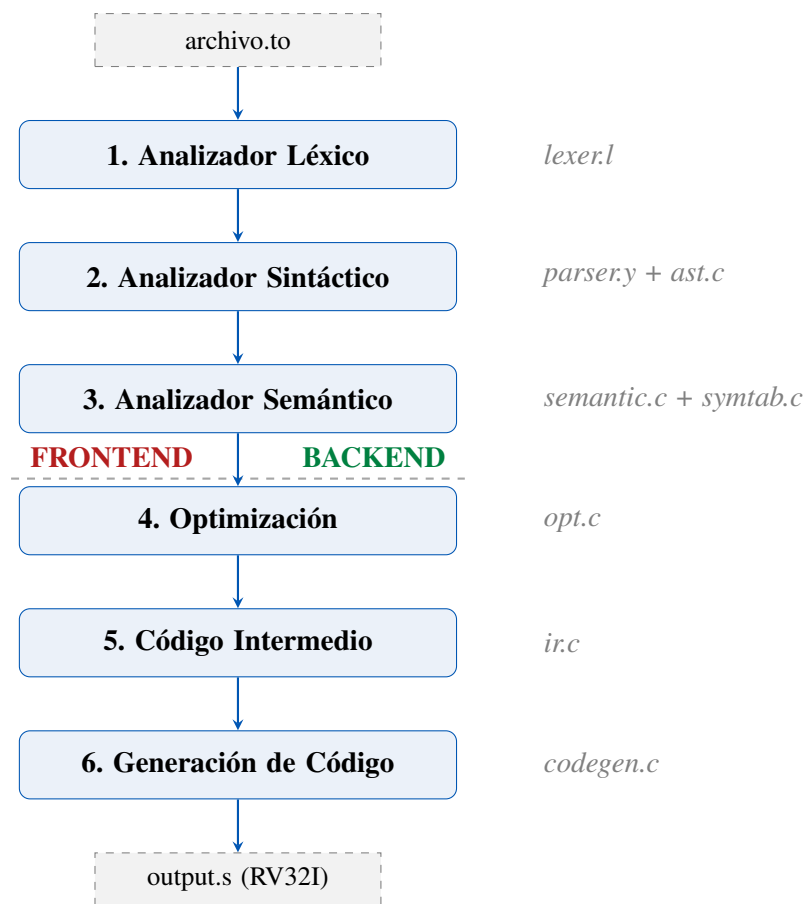


Figura 2.1: Pipeline completo del compilador TPP

2.3 Características del Lenguaje TPP

El lenguaje TPP soporta las siguientes características:

Categoría	Construcción	Ejemplo
Tipos de datos	Entero, Decimal	entero x = 5;
Arreglos	Estáticos de tamaño fijo	entero arr[10];
Control de flujo	si/sino, mientras, para	si (x > 0) { ... }
Funciones	Con parámetros y retorno	fun sumar(entero a) -> entero
E/S	Imprimir y leer	imprimir("Hola");
Comentarios	Línea simple	// comentario
Operadores	Aritméticos y relacionales	+, -, *, /, ==, !=, <, >

Cuadro 2.1: Características del lenguaje TPP

2.4 Arquitectura Objetivo: RISC-V RV32I

El código generado se dirige a la ISA **RISC-V RV32I** (32-bit Integer), ejecutado en un procesador de ciclo único dentro de Logisim. Las características relevantes son:

- **Registros:** 32 registros de propósito general (x0–x31).
- **Stack Pointer:** Registro sp (x2) para manejo del stack.
- **Registros temporales:** t0–t6 para evaluación de expresiones.
- **Registros de argumentos:** a0–a7 para paso de parámetros.
- **TTY I/O:** La dirección 0xff000004 es el puerto de salida para imprimir.
- **Multiplicación:** Implementada por software (`__soft_mul`) ya que RV32I base no incluye mul.

2.5 Estructura de Archivos del Proyecto

Archivo	Líneas	Función
lexer.l	85	Análisis léxico (flex)
parser.y	321	Análisis sintáctico (bison) + orquestación
ast.c / ast.h	488 / 96	Árbol Sintáctico Abstracto
semantic.c	323	Análisis semántico y tabla de símbolos
syntab.c / syntab.h	141 / 30	Tabla de símbolos
opt.c	298	Optimizaciones del AST
ir.c	178	Código intermedio (TAC)
codegen.c	457	Generación de código RV32I

Cuadro 2.2: Archivos del compilador TPP

Capítulo 3 FRONTEND — Análisis Léxico

3.1 Fundamentos Teóricos

El **análisis léxico** es la primera fase del proceso de compilación. En esta etapa, el compilador lee el código fuente (una cadena de caracteres) y lo divide en unidades lógicas con significado propio llamadas **tokens**.

Teóricamente, el análisis léxico se basa en la teoría de **Lenguajes Regulares**. Cada categoría de token (como un identificador, un número o una palabra reservada) se describe mediante una **Expresión Regular (Regex)**. Para reconocer estas expresiones en el texto de entrada, el analizador construye un **Autómata Finito Determinista (DFA)**. El DFA consume la entrada carácter por carácter y transita entre estados hasta alcanzar un estado de aceptación, logrando aislar y clasificar la subcadena leída (el *lexema*).

¿Por qué es necesario el Análisis Léxico?

La separación entre análisis léxico y sintáctico es crucial por tres motivos:

1. **Eficiencia:** *Aislar tareas de bajo nivel (como descartar espacios en blanco y comentarios) permite procesarlas muy rápido, reduciendo la carga del parser sintáctico.*
2. **Simplicidad:** *El parser sintáctico trabaja con unidades abstractas (tokens) en lugar de lidiar con caracteres crudos.*
3. **Portabilidad:** *Las reglas específicas de codificación de caracteres y saltos de línea quedan contenidas en el lexer.*

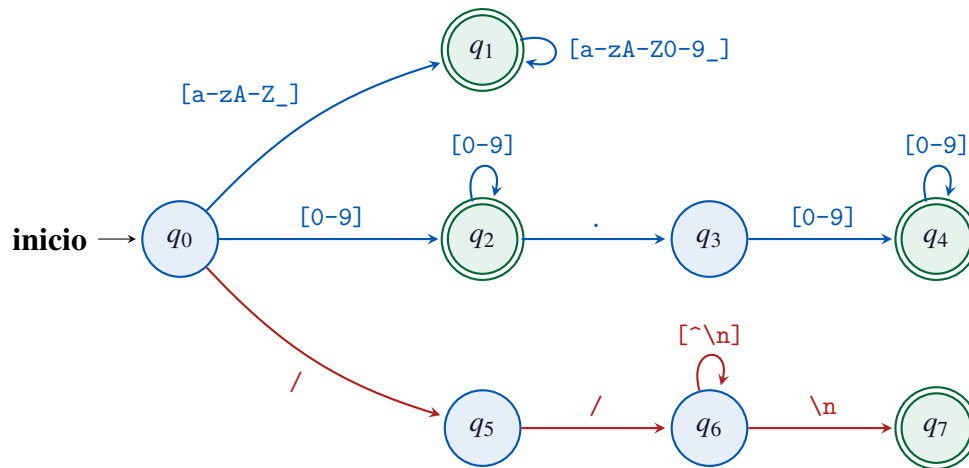


Figura 3.1: **Autómata Finito Determinista (DFA) del Lexer de TPP.** Los estados con doble borde (**verde**) son **estados de aceptación**. Rama superior (**azul**): $q_0 \rightarrow q_1$, identificador o palabra reservada. Rama media: $q_0 \rightarrow q_2$, entero; $q_2 \rightarrow q_3 \rightarrow q_4$, decimal. Rama inferior (**rojo**): $q_0 \rightarrow q_5 \rightarrow q_6 \rightarrow q_7$, comentario de línea.

3.2 Implementación con Flex: `lexer.1`

En el compilador TPP, el análisis léxico se ha delegado a **Flex** (*Fast Lexical Analyzer Generator*). Flex lee el archivo `lexer.1`, compila las expresiones regulares internamente convirtiéndolas primero en un Autómata Finito No Determinista (NFA) mediante la construcción de Thompson, y luego lo convierte a un DFA equivalente optimizado. Este DFA se plasma en código C (el archivo `lex.yy.c`) que exporta la función `yylex()`.

3.2.1 Configuración y Expresiones Regulares Base

El archivo inicia con código C que será inyectado directamente en el analizador generado, seguido de la declaración de macros de Flex:

```

1 %{
2     #include "parser.tab.h"
3     #include <stdio.h>
4     #include <stdlib.h>
5     #include <string.h>
6
7     // Contador de columna para reportar errores precisamente
8     int columna = 1;
9
10    // Acci n ejecutada ANTES de cada token: avanza el
11    // contador de columna
12    #define YY_USER_ACTION columna += yyleng;
13    %}
  
```

```

14 %option yylineno      // Habilita el conteo automatico de saltos
    de linea (\n)
15 %option noyywrap      // No encadenar multiples archivos fuente
16
17 // Expresiones regulares base (macros)
18 DIGITO [0-9]
19 LETRA  [a-zA-Z]

```

Listing 3.1: Sección de configuración del lexer

La directiva `YY_USER_ACTION` es esencial para el rastreo de errores: antes de que Flex ejecute la acción de cualquier regla, esta directiva actualiza la posición actual de la columna sumando `yylength` (la longitud del lexema detectado).

3.2.2 Reglas de Producción Léxica

Las reglas léxicas definen el patrón a buscar y el código C a ejecutar. Flex utiliza el principio de **Longitud Máxima (Maximal Munch)**: intentará leer tantos caracteres como sea posible mientras la expresión regular siga coincidiendo. Si dos reglas coinciden con la misma longitud, se prefiere la regla que fue definida primero.

```

1 // 1. PALABRAS RESERVADAS (Deben ir primero por el principio de
    prioridad)
2 "fun"      { return FUN; }
3 "si"       { return SI; }
4 "sino"     { return SINO; }
5
6 // 2. NÚMEROS (Uso del cierre positivo + para dígitos)
7 {DIGITO}+  { yylval.num_entero = atoi(yytext);
8              return NUM_ENTERO; }
9 {DIGITO}+"."{DIGITO}+ { yylval.num_decimal = atof(yytext);
10                        return NUM_DECIMAL; }
11
12 // 3. IDENTIFICADORES (Una letra seguida de 0 o más letras o
    números)
13 {LETRA}({LETRA}|{DIGITO})* { yylval.cadena = strdup(yytext);
14                             return ID; }

```

Listing 3.2: Reglas para palabras reservadas e identificadores

Nota Técnica

El atributo `yylval` es el puente de datos entre el Lexer y el Parser (definido como un *union* en Bison). Cuando reconocemos un número o un identificador, convertimos el lexema bruto `yytext` (un arreglo de `char`) a su valor correspondiente numérico o creamos una copia en memoria usando `strdup`, y lo almacenamos en `yylval` antes de retornar el token.

3.2.3 Tabla Completa de Tokens

Categoría	Patrón Regular	Acción (Token/Ignorar)
Palabras reservadas	"fun", "si", "sino"...	FUN, SI, SINO...
Literales Enteros	[0-9]+	NUM_ENTERO
Literales Decimales	[0-9]+.[0-9]+	NUM_DECIMAL
Cadenas de texto	"([^\"] \\.)*"	CADENA
Identificadores	[a-zA-Z][a-zA-Z0-9]*	ID
Operadores aritm.	+, -, *, /, %	MAS, MENOS, MULT, DIV, MOD
Asignación	=	ASIG
Operadores relac.	==, !=, <, >, <=, >=	IGUAL, DIFERENTE, ...
Puntero de retorno	->	FLECHA
Delimitadores	(,), {, }, [,]	PARI, PARD, LLAVEI, LLAVED...
Puntuación	;, :	PUNTOCOMA, COMA, DOS_PUNTOS
Comentarios	//.*	<i>Ignorado (skip)</i>
Espacios	[\t\n\r]+	<i>Ignorado (skip)</i>

Cuadro 3.2: Mapeo de expresiones regulares a Tokens en TPP

3.3 Detección y Manejo de Errores Léxicos

Una característica robusta del analizador léxico de TPP es su capacidad de recuperarse ante caracteres no reconocidos. Se utiliza la regla universal de un punto (.) al final del archivo `lexer.l`. Puesto que Flex da prioridad a las reglas definidas antes o a las de mayor longitud, esta regla de *fallback* o comodín solo atrapa caracteres inválidos (por ejemplo @, # o _).

```
1 . {
2     fprintf(stderr,
3         "Error L xico en la l nea %d, columna %d: Car cter
4         inv lido '%s'\n",
5         yylineno, column - (int)yytext);
}
```

Listing 3.3: Regla general de manejo de errores léxicos

Esta técnica permite al compilador advertir al programador con gran precisión posicional sin colgarse abruptamente, permitiendo eventualmente al parser descubrir errores de cascada en caso de omisiones léxicas severas.

Capítulo 4 FRONTEND — Análisis Sintáctico

4.1 Fundamentos Teóricos

El **análisis sintáctico** (o *parsing*) es la segunda etapa de la compilación. A diferencia del lexer, que agrupa caracteres en tokens planos, el parser descubre la **estructura jerárquica** del programa. Comprueba si la secuencia lineal de tokens obedece a las reglas estructurales del lenguaje.

Teóricamente, el lenguaje se modela mediante una **Gramática Libre de Contexto (CFG)**, definida por:

- **Terminales:** Los tokens provenientes del lexer.
- **No Terminales:** Variables sintácticas (ej. *expresion*, *declaracion*).
- **Producciones:** Reglas de reemplazo (*No Terminal* $\rightarrow$ *símbolos*).
- **Símbolo Inicial:** El punto de partida (ej. *programa*).

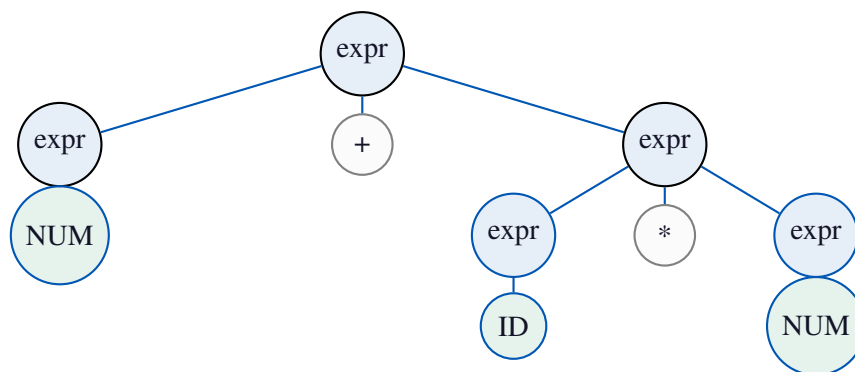


Figura 4.1: Ejemplo de Árbol de Derivación (Parse Tree) para la expresión matemática $3 + x * 5$, resolviendo la precedencia mediante la gramática.

¿Por qué es necesario el Análisis Sintáctico?

Una gramática regular (usada en el lexer) no puede contar ni anidar estructuras (no puede verificar que los paréntesis o llaves estén correctamente balanceados). Una CFG sí puede hacerlo. El análisis sintáctico permite:

1. *Validar que la sintaxis general es correcta (ej. un `si` seguido de `()` y `{}`).*
2. *Capturar precedencia y asociatividad de operadores.*
3. *Construir una representación en memoria abstracta (AST) para manipular el programa más fácilmente en el futuro.*

4.2 Implementación con Bison: `parser.y`

Para construir el parser de TPP se emplea **Bison**, que es un generador de parsers de tipo **LALR(1)** (*Look-Ahead Left-to-Right, Rightmost derivation with 1 token lookahead*). Bison toma la CFG escrita en `parser.y` y genera una máquina de estados (Pushdown Automaton) en C (`parser.tab.c`).

4.2.1 Funcionamiento Interno (Shift/Reduce)

El parser generado por Bison funciona mediante dos operaciones principales en una pila:

- **Shift (Desplazamiento):** Lee un token de la entrada y lo empuja a la pila sintáctica, pasando a un nuevo estado.
- **Reduce (Reducción):** Cuando la cima de la pila coincide con el lado derecho de una producción, saca esos elementos de la pila y empuja el lado izquierdo (el No Terminal). En este momento se ejecuta el código C asociado (la acción semántica), que en nuestro caso **crea un nodo del AST**.

4.2.2 Gramática del Lenguaje TPP

La gramática completa se define mediante reglas BNF en el archivo `parser.y`:

```

1 // Punto de entrada
2 programa:
3     lista_elementos {
4         NodoAST *raiz = nuevo_nodo_programa($1);
5         // Disparar el pipeline completo de compilación
6         analizar_semantica(raiz);
7         optimizar_ast(raiz);
8         generar_codigo_intermedio(raiz);
9         generar_codigo(raiz, out);
10    }
```

```

11
12 // Declaración de función
13 declaracion_fun:
14     FUN ID PARI parametros PARD FLECHA tipo LLAVEI
15     bloque_instrucciones LLAVED
16     { $$ = nuevo_nodo_declaracion_fun($2, $7, $4, $9); }
17
18     | FUN ID PARI parametros PARD LLAVEI
19     bloque_instrucciones LLAVED
20     { $$ = nuevo_nodo_declaracion_fun($2, 0, $4, $7); }
21
22 // Declaración de variable con tipo
23 declaracion_var:
24     tipo lista_ids PUNTOCOMA
25     { $$ = nuevo_nodo_declaracion_var($1, $2); }
26
27 // Estructura si/sino
28 estructura_si:
29     SI PARI expresion PARD LLAVEI bloque_instrucciones LLAVED
30     { $$ = nuevo_nodo_si($3, $6, NULL); } %prec LOWER_THAN_SINO
31
32     | SI PARI expresion PARD LLAVEI bloque_instrucciones LLAVED
33     SINO LLAVEI bloque_instrucciones LLAVED
34     { $$ = nuevo_nodo_si($3, $6, $10); }
35
36 // Llamada a función
37 llamada_funcion:
38     ID PARI argumentos PARD
39     { $$ = nuevo_nodo_llamada_funcion($1, $3); }
40
41 // Expresiones con precedencia
42 expresion:
43     expresion MAS expresion { $$ = nuevo_nodo_operacion(MAS,
44     $1, $3); }
45     | expresion MENOS expresion { $$ = nuevo_nodo_operacion(
46     MENOS, $1, $3); }
47     | expresion MULT expresion { $$ = nuevo_nodo_operacion(
48     MULT, $1, $3); }
49     | expresion DIV expresion { $$ = nuevo_nodo_operacion(DIV
50     , $1, $3); }
51     | expresion IGUAL expresion { $$ = nuevo_nodo_operacion(
52     IGUAL, $1, $3); }
53     | expresion MENOR expresion { $$ = nuevo_nodo_operacion(
54     MENOR, $1, $3); }
55     | NUM_ENTERO { $$ = nuevo_nodo_entero($1); }
56     | NUM_DECIMAL { $$ = nuevo_nodo_decimal($1); }
57     | ID { $$ = nuevo_nodo_variable($1); }
58     | CADENA { $$ = nuevo_nodo_cadena($1); }
59     | llamada_funcion { $$ = $1; }

```

Listing 4.1: Reglas gramaticales principales

4.2.3 Resolución de Ambigüedades

Bison resuelve ambigüedades mediante reglas de precedencia declaradas explícitamente:

```
1 // Menor a mayor precedencia (de arriba hacia abajo)
2 %left IGUAL DIFERENTE MENOR MAYOR MENOR_IGUAL MAYOR_IGUAL
3 %left MAS MENOS
4 %left MULT DIV MOD
5
6 // Resolución del "dangling else" (sino hu rfano)
7 %nonassoc LOWER_THAN_SINO
8 %nonassoc SINO
```

Listing 4.2: Declaraciones de precedencia

Esto garantiza que $3 + 4 * 2$ se evalúe como $3 + (4 * 2) = 11$ y no como $(3 + 4) * 2 = 14$.

4.3 Estructuras Sintácticas Soportadas

4.3.1 Declaraciones de Variables

```
1 entero x; // Simple sin inicializacion
2 entero x = 5; // Con inicializacion
3 entero x = 3 + 2; // Con expresion
4 entero arr[10]; // Arreglo de 10 enteros
5 decimal pi = 3.14; // Variable decimal
```

Listing 4.3: Formas válidas de declaración

4.3.2 Declaraciones de Funciones

```
1 // Funci n sin retorno (void)
2 fun saludar() {
3     imprimir("Hola");
4 }
5
6 // Funci n con par metros y tipo de retorno
7 fun sumar(entero a, entero b) -> entero {
8     retornar a + b;
9 }
10
11 // Funci n main (punto de entrada del usuario)
12 fun main() {
13     entero resultado;
14     resultado = sumar(3, 4);
15     imprimir(resultado);
16 }
```

Listing 4.4: Formas de declaración de funciones

4.3.3 Estructuras de Control

```

1 // Condicional
2 si (x > 0) {
3     imprimir("positivo");
4 } sino {
5     imprimir("negativo o cero");
6 }
7
8 // Bucle mientras
9 mientras (i < 10) {
10     i = i + 1;
11 }
12
13 // Bucle para (con declaracion de variable en cabecera)
14 para(entero i = 0; i < 5; i = i + 1) {
15     imprimir(arr[i]);
16 }
17
18 // Selecci n m ltiple
19 depende (x) {
20     caso 1: { imprimir("uno"); }
21     caso 2: { imprimir("dos"); }
22     otros:  { imprimir("otro"); }
23 }

```

Listing 4.5: Estructuras de control disponibles

4.4 Manejo de Errores Sintácticos

El parser implementa **recuperación de errores** para continuar el análisis aunque encuentre errores:

```

1 estructura_si:
2     SI PARI expresion PARD LLAVEI bloque_instrucciones LLAVED {
3         ... }
4     | SI PARI error PARD LLAVEI bloque_instrucciones LLAVED {
5         yyerrok;
6         printf("=> [Panic Mode] Error en condicion del SI.\n");
7         $$ = NULL;
8     }
9
10 estructura_para:
11     PARA PARI tipo ID ASIG expresion PUNTOCOMA
12     expresion PUNTOCOMA ID ASIG expresion PARD

```

```
12     LLAVEI bloque_instrucciones LLAVED { ... }
13     | PARA PARI error PARD LLAVEI bloque_instrucciones LLAVED {
14         yyerrok;
15         $$ = NULL;
16     }
```

Listing 4.6: Recuperación de errores en el parser

La función `yyerror()` produce mensajes descriptivos con línea y columna:

¡Ups! Hay un problema sintáctico en tu código.

Línea 14, Columna 11

Detalle: syntax error cerca del elemento 'cinco'

4.5 Orquestación del Pipeline

Una vez que el parser construye el AST sin errores, el punto de entrada programa actúa como orquestador y dispara todas las fases sucesivas del compilador en secuencia:

```
1 programa:
2     lista_elementos {
3         NodoAST *raiz = nuevo_nodo_programa($1);
4
5         // Fase 1: Análisis Semántico
6         int errores_sem = analizar_semantica(raiz);
7
8         if (errores_sem == 1) { // éxito
9             // Fase 2: Optimización del AST
10            optimizar_ast(raiz);
11
12            // Fase 3: Código Intermedio
13            generar_codigo_intermedio(raiz);
14
15            // Fase 4: Generación de Código Final
16            FILE *out = fopen("output.s", "w");
17            generar_codigo(raiz, out);
18            fclose(out);
19        }
20    }
```

Listing 4.7: Orquestación del pipeline en `parser.y`

Capítulo 5 FRONTEND — Análisis Semántico y Tabla de Símbolos

5.1 Fundamentos Teóricos

Mientras que el analizador sintáctico asegura que el código está escrito correctamente según las reglas gramaticales, el **analizador semántico** verifica que el programa tenga un significado lógico y consistente.

Teóricamente, el análisis semántico se ocupa de hacer cumplir reglas dependientes del contexto (Sensibles al Contexto), las cuales no pueden ser expresadas en una Gramática Libre de Contexto. Sus dos responsabilidades principales son:

- **Gestión de Entornos (Scoping):** Garantizar que las variables y funciones sean declaradas antes de ser usadas, respetando sus ámbitos de visibilidad (variables globales vs. locales).
- **Sistema de Tipos (Type Checking):** Asegurar que las operaciones se apliquen a operandos compatibles (e.g., no sumar un arreglo con un decimal) y que los valores de retorno de funciones coincidan con sus firmas.

El rol de la Tabla de Símbolos

*La **Tabla de Símbolos** es la estructura de datos principal utilizada en esta fase. Funciona como un diccionario dinámico que mapea identificadores (nombres de variables o funciones) a sus atributos lógicos (tipo, tamaño, ámbito y dirección en memoria).*

Adicionalmente, en el compilador TPP, esta fase calcula y almacena estáticamente las **direcciones de memoria absolutas y relativas** directamente en los nodos del AST, liberando al generador de código de la carga de gestionar variables.

5.2 Tabla de Símbolos: `syntab.c`

5.2.1 Estructura del Símbolo

Cada símbolo en la tabla tiene la siguiente estructura:

```

1 typedef enum {
2     SYM_VAR,      // Variable o arreglo
3     SYM_FUN      // Función
4 } CategoriaSimbolo;
5
6 typedef struct Simbolo {
7     char *nombre;           // Nombre del identificador
8     CategoriaSimbolo categoria; // Variable o Función
9     int tipo;               // TIPO_ENTERO (268),
10    TIPO_DECIMAL (269), 0 (void)
11     int num_params;         // Cantidad de parámetros (
12    funciones)
13     int *tipos_params;      // Tipos de parámetros (
14    funciones)
15     int ambito;             // 0 = global, 1,2,3... =
16    local
17     int tamano;             // 1 para escalares, N para
18    arreglos de N elem.
19     unsigned int direccion_memoria; // Dirección en RAM (
20    offset desde sp)
21     struct Simbolo *sig;    // Lista enlazada
22 } Simbolo;

```

Listing 5.1: Estructura Simbolo en symtab.h

5.2.2 Gestión de Memoria

La tabla de símbolos implementa un modelo de memoria lineal creciente. Cada nueva variable recibe un offset consecutivo desde la posición 0:

```

1 static unsigned int mem_disponible = 0;
2
3 Simbolo* insertar_simbolo(char *nombre, CategoriaSimbolo cat,
4                          int tipo, int ambito, int tamano) {
5     Simbolo *nuevo = malloc(sizeof(Simbolo));
6     // ...configurar campos...
7
8     if (cat == SYM_VAR) {
9         nuevo->direccion_memoria = mem_disponible;
10        mem_disponible += 4 * tamano; // Cada word = 4 bytes
11    } else {
12        nuevo->direccion_memoria = 0; // Funciones no ocupan
13        RAM
14    }
15
16    nuevo->sig = tabla_simbolos; // Inserción al frente (LIFO)
17    tabla_simbolos = nuevo;

```

```

17     return nuevo;
18 }

```

Listing 5.2: Asignación de memoria en insertar_simbolo()

Nota Técnica

Las variables y arreglos se almacenan en el stack ($sp + offset$). Un arreglo entero `arr[5]` reserva $5 \times 4 = 20$ bytes consecutivos a partir de su dirección base.

5.2.3 Scoping y Eliminación de Ámbitos

El compilador implementa ámbitos anidados mediante un contador `ambito_actual`. Cuando se sale de una función o bloque, se eliminan los símbolos de ese ámbito de la tabla (pero sus direcciones de memoria ya están grabadas en el AST):

```

1  case N_DECLARACION_FUN: {
2      insertar_simbolo(nodo->nombre_var, SYM_FUN, ...);
3
4      // Reservar espacio para guardar 'ra' (return address)
5      Simbolo *ra_sym = insertar_simbolo(".ra_nombre", SYM_VAR,
6          ...);
7      nodo->val_entero = ra_sym->direccion_memoria; // Para el
8          pr logo
9
10     ambito_actual++;           // Nuevo mbito
11     visitar_nodo(nodo->izq);    // Parametros
12     visitar_nodo(nodo->der);    // Cuerpo
13
14     eliminar_ambito(ambito_actual); // Limpia variables locales
15     ambito_actual--;
16 }

```

Listing 5.3: Eliminación de ámbito al salir de función

5.3 Implementación: semantic.c

5.3.1 Anotación de Nodos del AST

La función `visitar_nodo()` recorre el AST anotando cada nodo con su dirección de memoria. Este proceso es crítico porque permite al backend trabajar sin tabla de símbolos:

```

1  case N_VARIABLE: {
2      Simbolo *sym = buscar_simbolo(nodo->nombre_var);
3      if (!sym || sym->categoria != SYM_VAR) {
4          reportar_error("Uso de variable no declarada:", nodo->
5              nombre_var);
6      } else {

```

```

6      // *** Clave: grabar la direcci n en el nodo para
      codegen ***
7      nodo->val_entero = sym->direccion_memoria;
8  }
9      break;
10 }
11
12 case N_ASIGNACION: {
13     Simbolo *sym = buscar_simbolo(nodo->nombre_var);
14     if (sym) {
15         nodo->val_entero = sym->direccion_memoria; // Para
            codegen
16         // ...verificaci n de tipos...
17     }
18     visitar_nodo(nodo->izq);
19     break;
20 }
21
22 case N_ACCESO_ARREGLO: {
23     Simbolo *sym = buscar_simbolo(nodo->nombre_var);
24     if (sym) {
25         nodo->val_entero = sym->direccion_memoria; // base del
            arreglo
26     }
27     visitar_nodo(nodo->izq); // Resolver tambi n el ndice
28     break;
29 }

```

Listing 5.4: Anotación de variables en el AST

5.3.2 Verificación de Tipos

```

1 static int obtener_tipo_expresion(NodoAST *nodo) {
2     switch (nodo->tipo) {
3         case N_ENTERO: return TIPO_ENTERO;
4         case N_DECIMAL: return TIPO_DECIMAL;
5         case N_VARIABLE: {
6             Simbolo *sym = buscar_simbolo(nodo->nombre_var);
7             if (sym) {
8                 nodo->val_entero = sym->direccion_memoria;
9                 return sym->tipo;
10            }
11            // ...error si no encontrado...
12        }
13        case N_OPERACION: {
14            int tipo_izq = obtener_tipo_expresion(nodo->izq);
15            int tipo_der = obtener_tipo_expresion(nodo->der);
16            // Si alguno es decimal, el resultado es decimal
17            if (tipo_izq == TIPO_DECIMAL || tipo_der ==
                TIPO_DECIMAL)

```

```
18         return TIPO_DECIMAL;
19     return TIPO_ENTERO;
20 }
21 }
22 }
```

Listing 5.5: Verificación de tipos en expresiones

5.4 Errores Semánticos Detectados

Error	Ejemplo	Mensaje
Variable no declarada	x = y + 1; (y sin declarar)	Error: Variable no declarada
Redefinición de símbolo	entero x; entero x;	Error: Redefinición del símbolo
Uso no-variable	fun() = 5;	Error: No es una variable
Función no declarada	z = foo(1); (foo sin definir)	Error: Función no declarada
Tipo de índice	arr[3.14];	Error: Índice debe ser entero

Cuadro 5.1: Errores semánticos detectados por el compilador

5.5 Visualización de la Tabla de Símbolos

Al final del análisis semántico, el compilador imprime la tabla de símbolos en pantalla con información legible:

```
===== TABLA DE SIMBOLOS =====
Nombre          Categoria  Tipo      Ambito  Tam      Memoria
-----
.ra_main         Variable  entero    0        1      0x00000014
main             Funcion   ninguno   0        -      0x00000000
.ra_duplicar     Variable  entero    0        1      0x00000008
duplicar         Funcion   entero    0        -      0x00000000
c                Variable  entero    0        1      0x00000004
a                Variable  entero    0        1      0x00000000
-----
Total: 6 simbolos | Memoria usada: 28 bytes (7 words)
=====
```

- Las entradas .ra_nombre son slots de memoria reservados para guardar el ra (return address) de cada función.

- **Tam** muestra [N] para arreglos de N elementos.
- **Memoria** es el offset desde el base del stack sp.

Capítulo 6 BACKEND — El Árbol Sintáctico Abstracto (AST)

6.1 Fundamentos Teóricos: AST vs Parse Tree

El **Árbol Sintáctico Abstracto (AST)** es la estructura de datos central que divide el Frontend del Backend en el compilador.

Teóricamente, durante el análisis sintáctico se forma implícitamente un *Parse Tree* (Árbol de Análisis Sintáctico Clásico), el cual contiene absolutamente todos los terminales y no terminales de la gramática (incluyendo paréntesis, corchetes, puntos y comas). Sin embargo, un Parse Tree es demasiado ineficiente para el análisis posterior.

El AST es una versión "resumida" y destilada de este árbol que:

- Preserva únicamente la **estructura lógica** y la precedencia matemática.
- Elimina tokens superfluos que solo sirvieron para guiar al parser (como `;`, `(`, `{`).
- Sirve como **interfaz universal**. Gracias al AST, si mañana deseamos que TPP compile a ARM en lugar de RISC-V, solo necesitamos reemplazar el Backend; y si deseamos cambiar la sintaxis a inglés, solo reemplazamos el Frontend. El AST se mantiene inmutable.

Para procesar el AST en el Backend, el compilador utiliza **Recorridos en Profundidad (DFS - Depth First Search)**. Una función recursiva (`visitar_nodo`) desciende primero hasta las hojas del árbol antes de procesar la raíz.

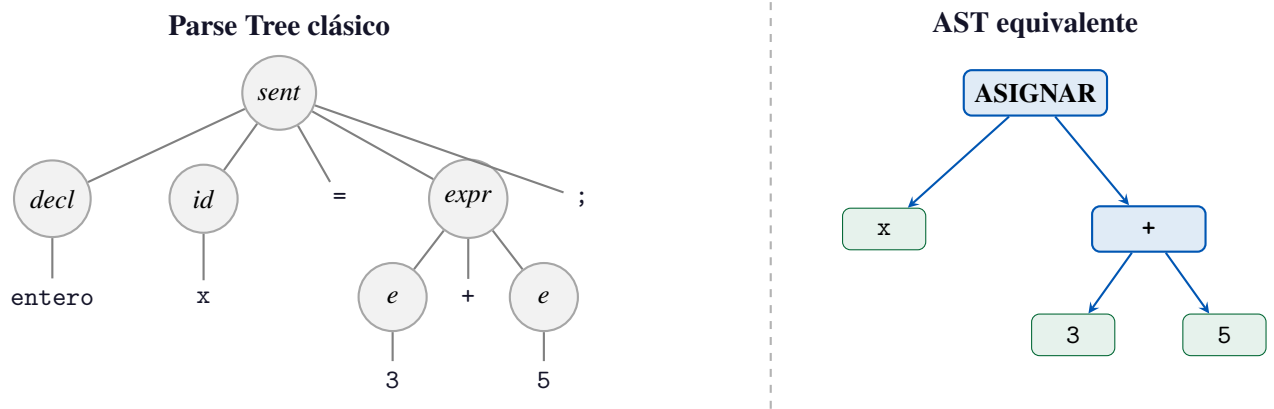


Figura 6.1: Comparación entre el **Parse Tree clásico** (izquierda) y el **AST generado por TPP** (derecha) para la sentencia `entero x = 3 + 5;`. El AST descarta nodos superfluos (`;`, `=`, tipo `entero`, nodos intermedios) y conserva únicamente la estructura lógica: asignación, variable y expresión sumada.

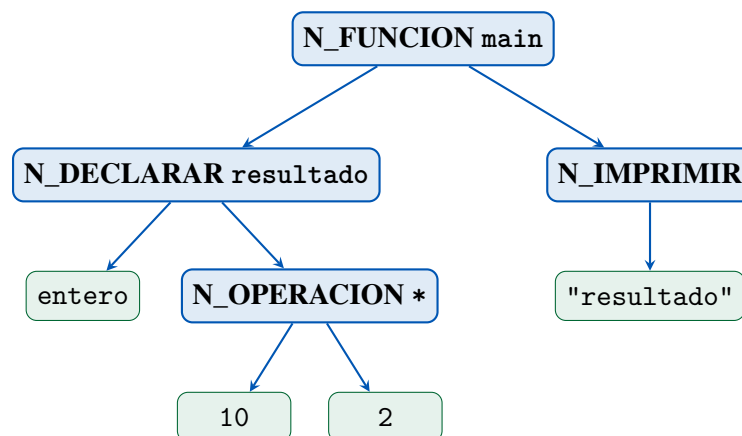


Figura 6.2: **AST generado por TPP** para `fun main(){ entero resultado = 10*2; imprimir(resultado); }`. Nodos azules = nodos internos con semántica; nodos verdes = hojas terminales (literales o identificadores).

6.2 Estructura del Nodo

Todos los nodos del AST comparten la misma estructura en memoria:

```
1 typedef struct NodoAST {
2     TipoNodo tipo;           // Discriminador: qu tipo de
    nodo es
3
4     // Ramas (hijos)
5     struct NodoAST *izq;     // Hijo izquierdo
6     struct NodoAST *der;     // Hijo derecho
```

```

7      struct NodoAST *centro; // Bloque SINO (si/sino) o paso (
      para)
8      struct NodoAST *adicional; // Cuerpo del para
9
10     // Campo multipropósito
11     int operador;           // Token del operador (+, -, <,
      etc.)
12                               // O tipo de la declaración (
      TIPO_ENTERO, etc.)
13
14     // Valores de hoja
15     int val_entero;         // Valor entero literal O
      direccion de memoria
16     double val_decimal;    // Valor decimal literal
17     char *nombre_var;      // Nombre de variable/función
18     char *val_cadena;      // Literal de cadena
19 } NodoAST;

```

Listing 6.1: Definición del nodo AST en ast.h

Nota Técnica

El campo `val_entero` tiene **doble propósito** : almacena el valor de literales enteros (`N_ENTERO`), y después del análisis semántico almacena la **dirección de memoria** (offset desde `sp`) para variables, arreglos y parámetros.

6.3 Tipos de Nodo

TipoNodo	Descripción	Hijos principales
N_PROGRAMA	Raíz del árbol	izq: lista elementos
N_LISTA_ELEMENTOS	Lista de instrucciones	izq, der: elementos
N_ENTERO	Literal entero	val_entero
N_DECIMAL	Literal decimal	val_decimal
N_CADENA	Literal cadena	val_cadena
N_VARIABLE	Uso de variable	nombre_var, val_entero (offset)
N_OPERACION	Operación binaria	izq, der, operador
N_ASIGNACION	Asignación	nombre_var, izq: expresión
N_DECLARACION_VAR	Declaración de variable	operador: tipo, izq: lista
N_DECLARACION_ARREGLO	Declaración de arreglo	nombre_var, val_entero: tamaño
N_ACCESO_ARREGLO	Acceso arr[i]	nombre_var, izq: índice
N_ASIGNACION_ARREGLO	Asignación arr[i]=v	nombre_var, izq: índice, der: valor
N_SI	Condicional si/sino	izq: cond, der: si, centro: sino
N_MIENTRAS	Bucle mientras	izq: cond, der: cuerpo
N_PARA	Bucle para	izq: init, der: cond, centro: paso, adicional: cuerpo
N_IMPRIMIR	Instrucción imprimir	izq: lista impresión
N_LEER	Instrucción leer	nombre_var
N_RETORNAR	Instrucción retornar	izq: expresión
N_DECLARACION_FUN	Declaración de función	nombre_var, operador: tipo retorno, izq: params, der: cuerpo
N_LLAMADA_FUNCION	Llamada a función	nombre_var, izq: argumentos
N_PARAMETRO	Parámetro formal	nombre_var, operador: tipo
N_LISTA_PARAMETROS	Lista de parámetros	izq, der
N_LISTA_ARGUMENTOS	Lista de argumentos	izq, der
N_DEPENDE	Switch/depende	izq: expr, der: casos
N_CASO	Caso dentro de depende	izq: valor, der: bloque

Cuadro 6.2: Tipos de nodos del AST

6.4 Ejemplo de AST

Para el programa:

```

1 entero x = 3 + 5;
2 imprimir(x);

```

El AST resultante es:

```

[Programa / Nodo Raiz]
| [Lista Elementos / Instrucciones]
| | [Declaracion Var] Tipo: 'entero'
| | | [Asignacion] a variable 'x'
| | | | [Operacion] '+'
| | | | | [Entero] 3
| | | | | [Entero] 5
| | [Imprimir]
| | | [Variable] x

```

Después del **plegado de constantes**, el nodo N_OPERACION se convierte directamente en [Entero] 8.

6.5 Construcción de Nodos

Cada tipo de nodo tiene su función constructora en `ast.c`:

```

1 NodoAST *nuevo_nodo_entero(int valor) {
2     NodoAST *nodo = crear_nodo(N_ENTERO);
3     nodo->val_entero = valor;
4     return nodo;
5 }
6
7 NodoAST *nuevo_nodo_operacion(int operador, NodoAST *izq,
8     NodoAST *der) {
9     NodoAST *nodo = crear_nodo(N_OPERACION);
10    nodo->operador = operador;
11    nodo->izq = izq;
12    nodo->der = der;
13    return nodo;
14 }
15
16 NodoAST *nuevo_nodo_declaracion_fun(char *nombre, int
17     tipo_retorno,
18                                     NodoAST *parametros,
19                                     NodoAST *bloque) {
20     NodoAST *nodo = crear_nodo(N_DECLARACION_FUN);
21     nodo->nombre_var = strdup(nombre);
22     nodo->operador = tipo_retorno; // 0 = void, 268 = entero,
    etc.
23     nodo->izq = parametros;
24     nodo->der = bloque;
25     return nodo;

```

23 }

Listing 6.2: Funciones constructoras de nodos del AST

Capítulo 7 BACKEND — Optimización del AST

7.1 Fundamentos Teóricos

La fase de **optimización** (Middle-end) tiene como objetivo transformar el programa en uno equivalente pero más eficiente (en velocidad de ejecución y uso de memoria). En TPP, las optimizaciones se realizan sobre el AST de manera independiente de la máquina (Machine-Independent Optimizations).

Teóricamente, las optimizaciones preservan la **semántica observacional** del programa: si se introducen los mismos datos de entrada, debe producir los mismos resultados y tener los mismos efectos secundarios.

Mecánica Interna en C

En la implementación en C (`opt.c`), las optimizaciones de AST implican:

1. **Recorrido Post-Orden (Bottom-up):** El optimizador utiliza recursión para llegar primero a las hojas del AST y luego subir. Esto asegura que al intentar optimizar un nodo padre, sus hijos ya estén completamente reducidos.
2. **Reemplazo en memoria:** Cuando un nodo (ej. `N_OPERACION`) es optimizado a un escalar, el puntero del nodo se sobrescribe modificando su discriminador (`tipo`) a `N_ENTERO`, evitando reestructurar agresivamente los punteros de todo el árbol.

7.1.1 Justificación del Orden de Ejecución

En la arquitectura del compilador TPP (contrario a la mayoría de compiladores modernos), la **optimización del AST se ejecuta estrictamente antes que la generación del Código Intermedio (TAC)**. Esta decisión de diseño se fundamenta en tres razones principales:

1. **Facilidad de manipulación matemática:** Es computacionalmente más sencillo recorrer un árbol para identificar y evaluar expresiones constantes (como `3 + 4`) que realizar el mismo análisis sobre instrucciones aplanadas y linealizadas.

2. **Reducción del tamaño del TAC:** Al optimizar y plegar constantes en el AST, el árbol resultante tiene menos nodos. Como consecuencia directa, el generador de TAC emitirá muchas menos cuádruplas y no desperdiciará registros temporales en cálculos redundantes.
3. **Desacoplamiento arquitectónico:** Realizar optimizaciones independientes de la máquina a nivel del AST garantiza que cualquier *Backend* posterior (sea RV32I u otra arquitectura) reciba un insumo semánticamente simplificado, delegando al Backend solo las optimizaciones dependientes de hardware (como asignación de registros físicos).

El compilador TPP implementa cuatro optimizaciones clásicas directamente sobre el AST:

Entrada y Salida del Optimizador

Entrada: AST anotado con direcciones de memoria

Salida: AST transformado con expresiones simplificadas y código muerto eliminado

El optimizador trabaja en un recorrido **bottom-up** (de hojas hacia la raíz), lo que permite que los resultados de los hijos estén ya simplificados cuando se procesa el nodo padre.

7.2 Optimización 1: Plegado de Constantes

7.2.1 Descripción

El **plegado de constantes** (*constant folding*) evalúa en tiempo de compilación todas las operaciones cuyos operandos son literales constantes. El nodo `N_OPERACION` se reemplaza directamente por un `N_ENTERO` con el resultado.

7.2.2 Operaciones soportadas

Operador	Antes	Después
+	3 + 5	8
-	10 - 3	7
*	4 * 5	20
/	10 / 2	5
==	3 == 3	1
!=	3 != 4	1
<	2 < 5	1
>	5 > 2	1

Cuadro 7.1: Operaciones sujetas a plegado de constantes

7.2.3 Implementación

```

1 static void plegado_constantes(NodoAST *nodo) {
2     if (nodo->tipo != N_OPERACION || !nodo->izq || !nodo->der)
3         return;
4     if (nodo->izq->tipo != N_ENTERO || nodo->der->tipo !=
5         N_ENTERO) return;
6
7     int a = nodo->izq->val_entero;
8     int b = nodo->der->val_entero;
9     int res = 0, valido = 1;
10
11     switch (nodo->operador) {
12         case MAS:         res = a + b; break;
13         case MENOS:        res = a - b; break;
14         case MULT:         res = a * b; break;
15         case DIV:          if (b != 0) res = a / b; else valido
16                             = 0; break;
17         case IGUAL:         res = (a == b); break;
18         case DIFERENTE:    res = (a != b); break;
19         case MENOR:        res = (a < b); break;
20         case MAYOR:        res = (a > b); break;
21         case MENOR_IGUAL:  res = (a <= b); break;
22         case MAYOR_IGUAL:  res = (a >= b); break;
23         default:           valido = 0; break;
24     }
25
26     if (valido) {
27         // Convertir el nodo operacion en un nodo entero
28         // constante
29         convertir_a_entero(nodo, res);
30         opt_const_fold++;
31     }
32 }

```

```
27     }
28 }
```

Listing 7.1: Implementación del plegado de constantes en opt.c



Figura 7.1: Visualización del Plegado de Constantes. El subárbol de la operación constante (3 + 5) colapsa en un solo nodo hoja constante (8).

7.2.4 Ejemplo

```
1 entero r = 10 * 2 + 3;
2 // Paso 1: 10 * 2      20
3 // Paso 2: 20 + 3      23
4 // Resultado: entero r = 23;
```

Listing 7.2: Plegado de constantes en cascada

7.3 Optimización 2: Simplificación Algebraica

7.3.1 Descripción

La **simplificación algebraica** aplica identidades matemáticas para eliminar operaciones innecesarias que involucren los elementos neutros y absorbentes.

Patrón	Ejemplo	Resultado	Identidad
$x + 0$	resultado + 0	resultado	Neutro de la suma
$0 + x$	0 + resultado	resultado	Neutro de la suma
$x - 0$	contador - 0	contador	Neutro de la resta
$x * 1$	valor * 1	valor	Neutro de la multiplicación
$1 * x$	1 * valor	valor	Neutro de la multiplicación
$x * 0$	n * 0	0	Absorbente de la multiplicación
$0 * x$	0 * n	0	Absorbente de la multiplicación
$x / 1$	n / 1	n	Neutro de la división

Cuadro 7.2: Identidades algebraicas aplicadas



Figura 7.2: Visualización de la Simplificación Algebraica. La operación de suma con el elemento neutro 0 ($x + 0$) se elimina, promoviendo el nodo de la variable (x) para reemplazar a su padre.

7.4 Optimización 3: Eliminación de Código Muerto

7.4.1 Descripción

La **eliminación de código muerto** (*dead code elimination*) detecta bloques de código que **nunca pueden ejecutarse** y los elimina del AST.

7.4.2 Casos detectados

```

1 // CASO 1: si(0) -> el bloque NUNCA se ejecuta -> eliminar
2 si (0) {
3     imprimir("Nunca se ejecuta");
4 }
5 // => Nodo eliminado completamente del AST
6
7 // CASO 2: si(1) -> SIEMPRE se ejecuta -> eliminar el si,
8 // quedar solo con bloque
9 si (1) {
10     imprimir("Siempre se ejecuta");
11 }
12 // => Se mantiene solo: imprimir("Siempre se ejecuta")
13
14 // CASO 3: mientras(0) -> bucle nunca se itera -> eliminar
15 mientras (0) {
16     imprimir("Nunca se ejecuta");
17 }
18 // => Nodo convertido en N_ENTERO 0 (inofensivo)
  
```

Listing 7.3: Casos de código muerto eliminados

7.5 Optimización 4: Propagación de Constantes

7.5.1 Descripción

La **propagación de constantes** (*constant propagation*) consiste en reemplazar el uso de una variable que siempre tiene un valor conocido en tiempo de compilación por ese valor directamente. En el compilador TPP, esta optimización ocurre de forma **implícita** a través de las pasadas repetidas del optimizador: cuando una expresión es plegada a una constante, los nodos que la usan también pueden ser plegados en la siguiente pasada.

7.6 Informe de Optimizaciones

Al finalizar la optimización, el compilador imprime un resumen:

```
===== OPTIMIZACION DE CODIGO =====
Optimizaciones aplicadas:
- Plegado de constantes:          4
- Simplificacion algebraica:     2
- Eliminacion de codigo muerto: 3
Total: 9 optimizaciones
```

7.7 Posibilidades de Optimización en el Código Final

La siguiente tabla muestra el impacto de las optimizaciones sobre el código ensamblador generado:

Optimización	Sin optimizar	Optimizado
Plegado 10 * 2 + 3	li t0, 10 call __soft_mul li t1, 3 add t0, t0, t1	li t0, 23
Algebraica x + 0	lw t0, N(sp) li t1, 0 add t0, t0, t1	lw t0, N(sp)
Código muerto si(0)	li t0, 0 beqz t0, .Lend [instrucciones] .Lend:	(eliminado)

Cuadro 7.3: Impacto de las optimizaciones en el código generado

Capítulo 8 BACKEND — Código Intermedio (TAC)

8.1 Fundamentos Teóricos

Una vez que el AST ha sido optimizado, el compilador necesita traducirlo a una forma más lineal que pueda ser procesada por fases sucesivas o por distintos backends. Esta forma intermedia es el **Código de Tres Direcciones (TAC, *Three-Address Code*)**.

Teóricamente, el TAC modela un tipo de máquina abstracta llamada **Máquina de Registros Infinitos**: tiene una cantidad ilimitada de variables temporales ($t_0, t_1, t_2, \dots$) que actúan como registros de cómputo. Esto permite *linearizar* cualquier expresión anidada del AST sin tener que preocuparse aún por la limitación de registros físicos de la CPU real.

Cada instrucción TAC puede expresarse formalmente como una **cuádrupla**:

$$(\text{op}, \text{arg}_1, \text{arg}_2, \text{result})$$

Por ejemplo:

- $t_2 = t_0 + t_1 \rightarrow (+, t_0, t_1, t_2)$
- $\text{if_false } t_2 \text{ goto } L1 \rightarrow (\text{if_false}, t_2, L1, -)$
- $\text{call duplicar} \rightarrow (\text{call}, \text{duplicar}, -, -)$

¿Por qué es necesario el Código Intermedio?

El TAC es un nivel de abstracción fundamental porque:

1. **Portabilidad:** Al ser independiente de la arquitectura (x86, ARM, RISC-V), el mismo TAC puede ser reutilizado para generar código para distintos procesadores simplemente intercambiando el backend.
2. **Diagnóstico:** Permite al programador o al docente leer y comprender qué operaciones realiza el compilador antes de sumergirse en el ensamblador.

3. **Base para optimizaciones posteriores:** Técnicas avanzadas como la Asignación Global de Registros (GRA), el Análisis de Flujo de Datos y la Eliminación de Subexpresiones Comunes (CSE) operan sobre formas de TAC o SSA.

Nota Técnica

En el compilador TPP, el código intermedio tiene propósito **diagnóstico y educativo**. No produce un archivo de salida propio; el TAC se imprime en pantalla para que el programador pueda entender cómo el compilador «ve» el programa antes de generar el ensamblador final.

8.2 Implementación: `ir.c`

8.2.1 Formato TAC

Cada instrucción TAC sigue uno de estos formatos:

```
t0 = 10           // Asignacion de constante a temporal
t1 = t0 + t2      // Operacion binaria
x = t1           // Asignacion a variable
arr[t1] = t0      // Escritura en arreglo
t0 = arr[t1]      // Lectura de arreglo
t0 = x < 5        // Comparacion
if t0 goto L1     // Salto condicional
goto L2           // Salto incondicional
L1:              // Etiqueta
call nombre_fun   // Llamada a función
ret              // Retorno
print "texto"     // Impresión de cadena
print t0          // Impresión de variable
```

8.2.2 Generador Recursivo

```
1 static int contador_temp = 0;
2 static int contador_etiq = 0;
3
4 void generar_tac(NodoAST *nodo) {
5     if (!nodo) return;
6
7     switch (nodo->tipo) {
8         case N_ENTERO:
```

```

9         printf("    t%d = %d\n", contador_temp++, nodo->
10             val_entero);
11         break;
12     case N_VARIABLE:
13         printf("    t%d = %s\n", contador_temp++, nodo->
14             nombre_var);
15         break;
16     case N_OPERACION: {
17         generar_tac(nodo->izq);
18         int temp_izq = contador_temp - 1;
19         generar_tac(nodo->der);
20         int temp_der = contador_temp - 1;
21         const char *op = obtener_nombre_operador(nodo->
22             operador);
23         printf("    t%d = t%d %s t%d\n",
24             contador_temp++, temp_izq, op, temp_der);
25         break;
26     }
27     case N_MIENTRAS: {
28         int etiq_inicio = contador_etiq++;
29         int etiq_fin = contador_etiq++;
30         printf("L%d:\n", etiq_inicio);
31         generar_tac(nodo->izq); // Condicion
32         printf("    t%d = %d\n", contador_temp,
33             contador_temp);
34         printf("    if_false t%d goto L%d\n", contador_temp
35             -1, etiq_fin);
36         generar_tac(nodo->der); // Cuerpo
37         printf("    goto L%d\n", etiq_inicio);
38         printf("L%d:\n", etiq_fin);
39         break;
40     }
41     case N_DECLARACION_FUN:
42         printf("fun %s:\n", nodo->nombre_var);
43         generar_tac(nodo->der); // Cuerpo
44         printf("end_fun %s\n", nodo->nombre_var);
45         break;
46     case N_LLAMADA_FUNCION:
47         printf("call %s\n", nodo->nombre_var);
48         printf("    t%d = t-1\n", contador_temp++); //
49             Valor de retorno
50         break;
51     case N_RETORNAR:
52         generar_tac(nodo->izq);
53         printf("ret\n");

```

```

54         break;
55     }
56 }

```

Listing 8.1: Generación de TAC recursiva en ir.c

8.3 Ejemplo de TAC

Para el programa:

```

1 entero x = 5;
2 entero y = x + 3;
3 imprimir(y);

```

El TAC generado sería:

```

t0 = 5
x = t0
t1 = x
t2 = 3
t3 = t1 + t2
y = t3
print y

```

Para una función:

```

fun duplicar:
    t0 = x
    t1 = 2
    t2 = t0 * t1
    res = t2
ret
end_fun duplicar

```

8.4 TAC para Estructuras de Control

8.4.1 Condicional si/sino

```

t0 = x
t1 = 0
t2 = t0 > t1          // t2 = condicion
if_false t2 goto L1   // Si falso, ir a L1 (sino)
// bloque si

```

```
    goto L2
L1:
    // bloque sino
L2:
```

8.4.2 Bucle para

```
    t0 = 0
    i = t0                // inicializacion: i = 0
L1:
    t1 = i
    t2 = 5
    t3 = t1 < t2          // condicion: i < 5
    if_false t3 goto L2
    // cuerpo del bucle
    t4 = i
    t5 = 1
    t6 = t4 + t5
    i = t6                // paso: i = i + 1
    goto L1
L2:
```

Capítulo 9 BACKEND — Generación de Código RV32I

9.1 Fundamentos Teóricos: Arquitectura RISC-V RV32I

La **generación de código** es la fase final y más concreta del compilador: traduce el AST optimizado a instrucciones de máquina reales para una arquitectura específica. En TPP, el objetivo es la ISA (*Instruction Set Architecture*) **RISC-V RV32I**.

RISC-V es una arquitectura de conjunto reducido de instrucciones (RISC) de código abierto. La variante **RV32I** es la base de 32 bits que incluye únicamente instrucciones enteras: cargas/almacenamientos (lw/sw), aritmética (add/sub), lógica, saltos condicionales (beqz/bnez) e incondicionales (j). No incluye multiplicación (mul), que forma parte de la extensión M.

9.1.1 Registros Arquitectónicos RV32I

Registro	Alias	Propósito (ABI)	Callee-Saved
x0	zero	Siempre cero	N/A
x1	ra	Return Address (dirección de retorno)	No
x2	sp	Stack Pointer	Sí
x5-x7	t0-t2	Temporales (uso general)	No
x10-x11	a0-a1	Argumentos/Retorno de función	No
x28-x31	t3-t6	Temporales adicionales	No

Cuadro 9.1: Registros utilizados por el compilador TPP en RV32I

9.1.2 Convención de Llamadas (Calling Convention)

La **convención de llamadas** define los contratos que el compilador debe respetar al invocar una función:

- Los **argumentos** se pasan en registros a0, a1, a2... en orden.

- El **valor de retorno** se deposita en `a0` antes de ejecutar `ret`.
- El **return address (ra)** es almacenado en la pila antes de llamar a una subfunción (para no perder el camino de vuelta).
- El **stack pointer (sp)** debe quedar exactamente en el mismo valor antes y después de la llamada (responsabilidad del *caller* o del *callee*, según el ABI).

En TPP, el compilador sigue un modelo simplificado: todas las variables tienen offsets fijos calculados en tiempo de compilación por `syntab.c`, eliminando la necesidad de ajustar `sp` en cada llamada.

Entrada y Salida del Generador de Código

Entrada: AST optimizado con nodos anotados con direcciones de memoria

Salida: Archivo `output.s` con instrucciones RISC-V RV32I ensamblables

9.2 Modelo de Memoria

9.2.1 Stack Único Compartido

El compilador TPP usa un modelo de memoria basado en un **stack único estático**. Todas las variables (globales, locales, parámetros de funciones) coexisten en el mismo stack frame, identificadas por su offset fijo desde `sp`:

```

sp+0  → variable global 'a' (primera declarada)
sp+4  → variable global 'c'
sp+8  → ra de función 'duplicar' (.ra_duplicar)
sp+12 → parámetro 'x' de duplicar
sp+16 → variable local 'res' de duplicar
sp+20 → ra de función 'main' (.ra_main)
sp+24 → variable local 'b' de main
sp+28 → ra del entry point
    
```

El stack pointer `sp` se inicializa en la RAM del procesador Logisim. La función de entrada `main`: reserva espacio para **todos** los offsets calculados por el análisis semántico:

```

1 main:
2     li    sp, 0x0FFC          # Inicializar SP al tope de RAM (4KB)
3     addi  sp, sp, -N          # N = memoria_total calculada por
        syntab
4     sw    ra, M(sp)          # Guardar return address del entry
        point
5     # Inicializar variables globales...
    
```



```

6      call __user_main      # Llamar al main del usuario
7      lw    ra, M(sp)
8      addi sp, sp, N
9  .halt_loop:
10     j .halt_loop          # Bucle infinito (halt del procesador)

```

Listing 9.1: Prólogo del entry point

9.3 Generación Bifásica para Funciones

Para evitar que el procesador ejecute el cuerpo de las funciones en caída libre después del `halt_loop`, el compilador implementa una **generación en dos pasadas**:

```

1 void generar_codigo(NodoAST *raiz, FILE *out) {
2     // PASADA 1: Emitir todo lo que NO es funci n
3     //     (inicializaciones globales, entry point, etc.)
4     generar_nodo_sin_funciones(raiz, out);
5
6     // PASADA 2: Emitir solo las declaraciones de funciones
7     //     (van DESPU S del halt_loop para que no se ejecuten
8     //         inline)
9     generar_solo_funciones(raiz, out);
10 }

```

Listing 9.2: Generación bifásica en `codegen.c`

El ensamblador resultante tiene esta estructura:

```

1 # ---- Rutinas de soporte ----
2 __print_str:  ...
3 __soft_mul:   ...
4 __print_num:  ...
5
6 # ---- Entry point ----
7 main:
8     addi sp, sp, -N
9     # Inicializar variables globales
10    call __user_main
11    lw ra, ...
12    addi sp, sp, N
13 .halt_loop:
14     j .halt_loop
15
16 # ---- Funciones del usuario (despu s del halt) ----
17 .globl mi_funcion
18 mi_funcion:
19     sw ra, R(sp)      # Pr logo: guardar ra
20     # Cuerpo de la funci n
21     lw ra, R(sp)      # Ep logo: restaurar ra
22     ret
23

```

```
24 .globl __user_main
25 __user_main:
26     sw ra, M(sp)
27     # Cuerpo del main del usuario
28     lw ra, M(sp)
29     ret
```

Listing 9.3: Estructura del output.s generado

9.4 Rutinas de Soporte Emitidas Automáticamente

El generador emite automáticamente tres rutinas de soporte al inicio del archivo:

Rutina	Propósito	Parámetros
__print_str	Imprime cadena en TTY Logisim	a0 = puntero a cadena
__soft_mul	Multiplicación por software	a0, a1 = factores
__print_num	Imprime entero como ASCII	a0 = número a imprimir

Cuadro 9.2: Rutinas de soporte generadas automáticamente

9.4.1 Multiplicador por software

Dado que la ISA RV32I base no incluye instrucción mul, el compilador implementa la multiplicación como una suma repetida:

```
1 __soft_mul:
2     li t0, 0          # acumulador = 0
3     li t1, 0          # contador = 0
4 .mul_loop:
5     beq t1, a1, .mul_end # si contador == a1, terminar
6     add t0, t0, a0      # acumulador += a0
7     addi t1, t1, 1
8     j .mul_loop
9 .mul_end:
10    mv a0, t0          # retornar resultado en a0
11    ret
```

Listing 9.4: Multiplicador por software: a0 = a0 * a1

9.5 Generación por Tipo de Nodo

9.5.1 Expresiones Aritméticas

La función traducir_expresion() genera código para cualquier expresión y deposita el resultado en un registro temporal:

```

1 static void traducir_expresion(NodoAST *nodo, FILE *out, int
   target_reg) {
2     switch (nodo->tipo) {
3         case N_ENTERO:
4             fprintf(out, "    li t%d, %d\n", target_reg, nodo->
               val_entero);
5             break;
6
7         case N_VARIABLE:
8             // val_entero contiene el offset desde sp (anotado
               por semantic.c)
9             fprintf(out, "    lw t%d, %d(sp)\n", target_reg,
               nodo->val_entero);
10            break;
11
12         case N_OPERACION: {
13             traducir_expresion(nodo->izq, out, target_reg);
14             traducir_expresion(nodo->der, out, target_reg + 1);
15             const char *op = obtener_nombre_operador(nodo->
               operador);
16
17             if (strcmp(op, "+") == 0)
18                 fprintf(out, "    add t%d, t%d, t%d\n",
19                     target_reg, target_reg, target_reg + 1)
20                 ;
21             else if (strcmp(op, "*") == 0) {
22                 // Multiplicaci n: usar __soft_mul
23                 fprintf(out, "    mv a0, t%d\n", target_reg);
24                 fprintf(out, "    mv a1, t%d\n", target_reg +
25                     1);
26                 fprintf(out, "    call __soft_mul\n");
27                 fprintf(out, "    mv t%d, a0\n", target_reg);
28             }
29             // ... otros operadores ...
30             break;
31         }
32
33         case N_ACCESO_ARREGLO: {
34             // val_entero = direcci n base del arreglo
35             traducir_expresion(nodo->izq, out, target_reg); //
               ndice
36             fprintf(out, "    slli t%d, t%d, 2\n", target_reg,
               target_reg);
37             fprintf(out, "    li t%d, %d\n", target_reg + 1,
               nodo->val_entero);
38             fprintf(out, "    add t%d, t%d, t%d\n",
               target_reg, target_reg, target_reg + 1);
39             fprintf(out, "    add t%d, t%d, sp\n", target_reg,
               target_reg);
40             fprintf(out, "    lw t%d, 0(t%d)\n", target_reg,
               target_reg);
41         }
42     }
43 }

```

```

40         break;
41     }
42 }
43 }

```

Listing 9.5: Generación de expresiones en codegen.c

9.5.2 Estructuras de Control

```

1  case N_MIENTRAS: {
2      int etiq = obtener_etiqueta();
3      fprintf(out, ".L%d:\n", etiq);
4
5      // Evaluar condici n
6      traducir_expresion(nodo->izq, out, 0);
7      fprintf(out, "    beqz t0, .L%d\n", etiq + 1);
8
9      // Cuerpo
10     generar_nodo(nodo->der, out);
11
12     fprintf(out, "    j .L%d\n", etiq);
13     fprintf(out, ".L%d:\n", etiq + 1);
14     break;
15 }

```

Listing 9.6: Generación del bucle mientras

```

1  case N_PARA: {
2      int etiq = obtener_etiqueta();
3
4      generar_nodo(nodo->izq, out);    // Inicializaci n
5      fprintf(out, ".L%d:\n", etiq);
6
7      traducir_expresion(nodo->der, out, 0); // Condici n
8      fprintf(out, "    beqz t0, .L%d\n", etiq + 1);
9
10     generar_nodo(nodo->adicional, out);    // Cuerpo
11     generar_nodo(nodo->centro, out);        // Paso (i = i + 1)
12
13     fprintf(out, "    j .L%d\n", etiq);
14     fprintf(out, ".L%d:\n", etiq + 1);
15     break;
16 }

```

Listing 9.7: Generación del bucle para

9.5.3 Funciones: Prólogo, Llamada y Epílogo

El manejo de funciones sigue la convención de llamada RISC-V:

```

1 case N_DECLARACION_FUN: {
2     fprintf(out, "        .globl %s\n%s:\n", nodo->nombre_var, nodo
        ->nombre_var);
3
4     // PR LOGO: guardar ra en su slot reservado (calculado por
        semantic.c)
5     int ra_offset = nodo->val_entero; // Offset del .ra_nombre
6     fprintf(out, "        sw ra, %d(sp)\n", ra_offset);
7
8     // Guardar par metros (a0, a1, ...) en sus slots
9     guardar_parametros(nodo->izq, out);
10
11    // Cuerpo de la funci n
12    generar_nodo(nodo->der, out);
13
14    // Etiqueta de fin para los retornar
15    fprintf(out, ".L_end_%s:\n", nodo->nombre_var);
16
17    // EP LOGO: restaurar ra y retornar
18    fprintf(out, "        lw ra, %d(sp)\n", ra_offset);
19    fprintf(out, "        ret\n\n");
20    break;
21 }

```

Listing 9.8: Generación de declaración de función

```

1 case N_LLAMADA_FUNCION: {
2     // Evaluar argumentos y cargarlos en a0, a1, ...
3     evaluar_argumentos(nodo->izq, out);
4
5     fprintf(out, "        call %s\n", nodo->nombre_var);
6
7     // El resultado queda en a0 (convenci n RISC-V)
8     fprintf(out, "        mv t0, a0\n");
9     break;
10 }

```

Listing 9.9: Generación de llamada a función

9.5.4 Instrucción imprimir

La instrucción imprimir detecta automáticamente el tipo de su argumento:

```

1 case N_IMPRIMIR: {
2     NodoAST *arg = nodo->izq;
3
4     if (arg->tipo == N_CADENA) {
5         // Cadena: emitir en .data y llamar __print_str
6         fprintf(out, "        .section .data\n");
7         fprintf(out, ".LSTR%d: .string %s\n", ++str_counter,
            arg->val_cadena);

```

```
8      fprintf(out, "      .section .text\n");
9      fprintf(out, "      lui a0, %%hi(.LSTR%d)\n", str_counter
10     );
11     fprintf(out, "      addi a0, a0, %%lo(.LSTR%d)\n",
12           str_counter);
13     fprintf(out, "      call __print_str\n");
14 } else {
15     // Entero/variable: evaluar y llamar __print_num
16     traducir_expresion(arg, out, 0);
17     fprintf(out, "      mv a0, t0\n");
18     fprintf(out, "      call __print_num\n");
19 }
20 // Salto de linea automatico
21 fprintf(out, "      li t0, 10\n");
22 fprintf(out, "      add zero, zero, t0 # FIX TTY Logisim\n");
23 ;
24 fprintf(out, "      li t1, 0xff000004\n");
25 fprintf(out, "      sw t0, 0(t1)\n");
26 break;
27 }
```

Listing 9.10: Generación de imprimir



Figura 9.1: Esquema de traducción post-orden. El generador primero emite el código para los hijos (izquierdo y derecho) almacenando sus resultados en registros temporales, y finalmente emite la instrucción que los combina.

9.6 Tabla de Posibilidades del Código Generado

Cuadro 9.3: Mapeo de construcciones TPP a instrucciones RV32I

Construcción TPP	Código RV32I generado	Notas
entero x = 5;	li t0, 5 sw t0, N(sp)	N = offset calculado por symtab

Continúa en la siguiente página

Cuadro 9.3 – Continuación de la página anterior

Construcción TPP	Código RV32I generado	Notas
<code>x = x + 1;</code>	<pre>lw t0, N(sp) li t1, 1 add t0, t0, t1 sw t0, N(sp)</pre>	Usa registros temporales t0, t1
<code>x = a * b;</code>	<pre>lw t0, N1(sp) lw t1, N2(sp) mv a0, t0 mv a1, t1 call __soft_mul mv t0, a0</pre>	Multiplicación por software
<code>arr[i] = v;</code>	<pre>lw t1, i_off(sp) slli t1, t1, 2 li t2, arr_base add t1, t1, t2 add t1, t1, sp sw t0, 0(t1)</pre>	Cálculo de dirección dinámico
<code>si(cond){...}</code>	<pre>[evaluar cond] beqz t0, .Lend [cuerpo] .Lend:</pre>	Salto sobre el bloque
<code>mientras(c){...}.L1:</code>	<pre>[evaluar c] beqz t0, .L2 [cuerpo] j .L1 .L2:</pre>	Bucle con etiquetas únicas
<code>imprimir("str")</code>	<pre>.section .data .LSTRN: .string "str" .section .text lui/addi a0 call __print_str</pre>	Cadena en sección .data

Continúa en la siguiente página

Cuadro 9.3 – Continuación de la página anterior

Construcción TPP	Código RV32I generado	Notas
imprimir(x)	lw t0, N(sp) mv a0, t0 call __print_num	Conversión int→ASCII
fun f(entero p){}	f: sw ra, R(sp) sw a0, P(sp) [cuerpo] .L_end_f: lw ra, R(sp) ret	Prólogo/epílogo completo
retornar expr;	[evaluar expr] mv a0, t0 j .L_end_nombre	Resultado en a0, salto al epílogo
b = f(a);	lw t0, a_off(sp) mv a0, t0 call f mv t0, a0 sw t0, b_off(sp)	Convención RISC-V estándar

9.7 Salida de Diagnóstico

El compilador imprime el contenido completo del output .s generado directamente en consola para facilitar la depuración:

```
===== CODIGO FINAL (RV32I) =====
.section .text

# ---- RUTINA: print_str ----
__print_str:
    li t1, 0xff000004
    ...

.globl main
main:
```



```
    addi sp, sp, -32
    sw ra, 28(sp)
    li t0, 10
    sw t0, 0(sp)
    call __user_main
.halt_loop:
    j .halt_loop
```

=====

Capítulo 10 METODOLOGÍA — Integración con Logisim Evolution

10.1 Visión General del Proceso

El compilador TPP no se detiene en la generación del archivo `output.s` (ensamblador RV32I en formato texto). Para que el programa pueda **ejecutarse realmente en el procesador** implementado en Logisim Evolution, es necesario un paso adicional: convertir el código ensamblador a **código máquina hexadecimal** cargable directamente en la memoria RAM del circuito.

Este proceso es realizado por el script `compile_to_logisim.py`, derivado y adaptado del repositorio público **riscv-asm** (disponible en GitHub), un ensamblador minimalista de RISC-V en Python diseñado específicamente para la carga de programas en simuladores de hardware educativos.

El flujo completo, desde el código fuente hasta la ejecución en el procesador, es el siguiente:

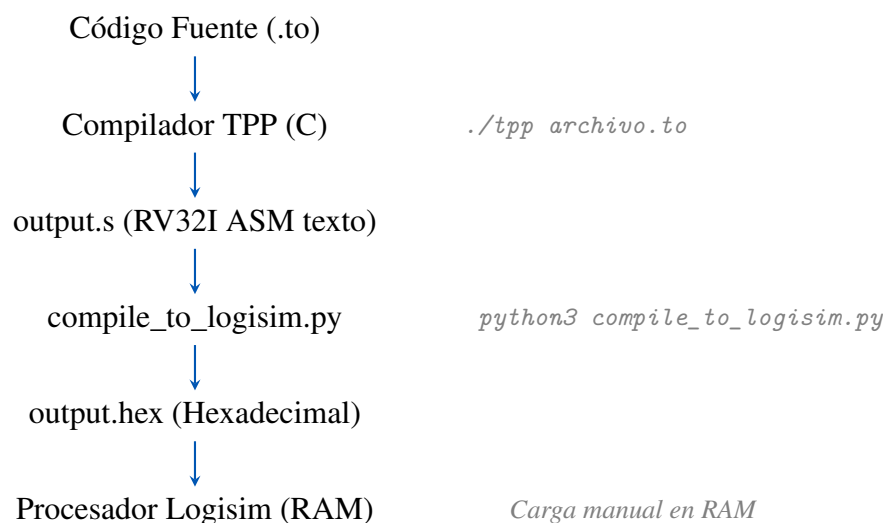


Figura 10.1: Pipeline completo hasta ejecución en Logisim

10.2 El Script `compile_to_logisim.py`

10.2.1 Origen y Adaptación

El script fue extraído y adaptado del repositorio `riscv-asm` disponible en GitHub. Este repositorio implementa un ensamblador RISC-V minimalista en Python, capaz de traducir código ensamblador RV32I a su representación binaria y hexadecimal, orientado a contextos educativos donde no se desea depender de toolchains complejos como `riscv64-unknown-elf-gcc`.

La adaptación realizada para TPP incluye:

- Soporte para el formato de memoria de Logisim Evolution (v2.0 raw).
- Manejo de etiquetas relativas (pseudo-instrucción `j`, `call`).
- Soporte de la sección `.data` para cadenas de texto literales.
- Traducción de la pseudo-instrucción `li` a su secuencia `lui + addi`.

10.2.2 Funcionamiento Interno

El script realiza los siguientes pasos en orden:

1. **Primera pasada — Recolección de etiquetas:** Lee el archivo `output.s` completo. Para cada línea que termina en `:` (ej. `main:`, `.L0:`), calcula la dirección de memoria correspondiente (contando words de 32 bits desde la dirección base `0x0000`) y registra la etiqueta en un diccionario de símbolos.
2. **Segunda pasada — Ensamblado:** Para cada instrucción reconocida, aplica la codificación de la ISA RV32I (formato R, I, S, B, U, J) para producir el **word de 32 bits** en hexadecimal.
 - Instrucciones de tipo R (`add`, `sub`, `and`): campos `funct7|rs2|rs1|funct3|rd|opcode`.
 - Instrucciones de tipo I (`lw`, `addi`): campo inmediato de 12 bits en complemento a dos.
 - Instrucciones de tipo S (`sw`): el inmediato se divide entre `inst[11:5]` e `inst[4:0]`.
 - Instrucciones de tipo B (`beqz`, `bne`): el inmediato codifica un offset en palabras con codificación especial.
 - Pseudo-instrucciones (`li`, `mv`, `j`, `call`, `ret`): se expanden a una o más instrucciones reales.
3. **Generación del archivo `output.hex`:** El archivo resultante inicia con la cabecera v2.0 raw (requerida por Logisim), seguido de un word hexadecimal por línea:

```
v2.0 raw
00000097
00000113
ff410113
...
```

4. **Carga en Logisim:** El archivo `output.hex` se carga en la componente RAM del circuito de Logisim mediante *Right Click* → *Load Image....* El procesador entonces lee las instrucciones desde la dirección `0x0000` al iniciar.

10.2.3 Codificación de Ejemplo

Para ilustrar el proceso, la instrucción `addi sp, sp, -32` se codifica de la siguiente manera en formato I:

imm[11:0]	rs1	funct3	rd	opcode	Hex
111111100000	00010	000	00010	0010011	fe010113
(= -32 en 12 bits)	(sp=x2)	(ADDI)	(sp=x2)	(OP-IMM)	

Cuadro 10.1: Codificación de `addi sp, sp, -32` en RV32I formato I

10.3 Arquitectura RISC-V y el Procesador Logisim

10.3.1 El Procesador de Ciclo Único

El procesador implementado en Logisim Evolution es un **procesador de ciclo único** (Single-Cycle Processor) para la ISA RISC-V RV32I. Sus componentes principales son:

- **Program Counter (PC):** Registro de 32 bits que apunta a la instrucción actual en RAM.
- **Memoria de Instrucciones (RAM):** Donde se carga el archivo `output.hex`.
- **Banco de Registros:** 32 registros de 32 bits (`x0–x31`). `x0` siempre es cero.
- **ALU:** Unidad Aritmético Lógica que ejecuta operaciones según el campo `funct3/funct7`.
- **Unidad de Control:** Decodifica el opcode y genera las señales de control para el datapath.
- **Memoria de Datos (RAM):** Accedida con instrucciones `lw/sw`.
- **TTY Output:** Dirección mapeada a memoria (`0xFF000004`) que actúa como terminal de salida de caracteres ASCII.

10.3.2 Ciclo de Ejecución por Instrucción

Cada instrucción se ejecuta en un único ciclo de reloj mediante los siguientes pasos del datapath:

1. **Fetch (IF):** Leer la instrucción de 32 bits desde RAM[PC].
2. **Decode (ID):** La unidad de control decodifica el opcode. Se leen los registros fuente.
3. **Execute (EX):** La ALU realiza la operación aritmética o calcula la dirección de memoria.
4. **Memory (MEM):** Si es lw/sw, se accede a la RAM de datos.
5. **Write Back (WB):** El resultado se escribe en el registro destino.
6. **PC Update:** $PC \leftarrow PC + 4$ (o $PC + \text{offset}$ para saltos).

Capítulo 11 COMPLICACIONES

11.1 Panorama General de Dificultades

La integración entre el compilador TPP (escrito en C) y el procesador Logisim Evolution no fue inmediata. A lo largo del desarrollo surgieron múltiples complicaciones técnicas en tres niveles distintos: la generación del código ensamblador, el proceso de ensamblado a hexadecimal, y la ejecución en el circuito simulado.

11.2 Complicación 1: Ausencia de Instrucción mul en RV32I Base

11.2.1 Descripción

La ISA RISC-V RV32I en su variante base (sin la extensión M) **no incluye instrucción de multiplicación**. El compilador TPP genera expresiones de multiplicación desde el lenguaje fuente, por lo que fue necesario encontrar una solución a nivel de generación de código.

11.2.2 Solución Adoptada

Se implementó una rutina de multiplicación por software (`__soft_mul`) que simula la multiplicación mediante sumas repetidas. Esta rutina es emitida automáticamente al inicio de cada archivo `output.s`:

```
1 __soft_mul:
2     li    t0, 0           # acumulador = 0
3     li    t1, 0           # contador = 0
4 .mul_loop:
5     beq    t1, a1, .mul_end # si contador == a1, terminar
6     add    t0, t0, a0       # acumulador += a0
7     addi   t1, t1, 1
8     j      .mul_loop
9 .mul_end:
10    mv     a0, t0           # retornar resultado en a0
11    ret
```

Listing 11.1: Rutina `__soft_mul` generada automáticamente**Nota Técnica**

Esta solución funciona correctamente para operandos positivos. La multiplicación de números negativos o de resultado mayor a `0x7FFFFFFF` no está soportada por esta implementación de suma repetida.

11.3 Complicación 2: Salida TTY en Logisim

11.3.1 Descripción

El mecanismo de salida por consola en Logisim Evolution difiere de la convención estándar de RISC-V. En el procesador estándar se usan syscalls (instrucciones `ecall`), pero Logisim no las implementa. En cambio, utiliza **Memoria Mapeada a I/O (MMIO)**: la dirección `0xFF000004` está conectada directamente al componente TTY del circuito.

11.3.2 Problema de Codificación

La dirección `0xFF000004` es un valor de 32 bits que no cabe en un inmediato de 12 bits (el límite de `addi`). Se requiere la instrucción `lui` (Load Upper Immediate) para cargar los 20 bits superiores, seguida de `addi` para los 12 inferiores. Sin embargo, el bit de signo del inmediato de `addi` causa que valores superiores a `0x7FF` sean extendidos en signo, requiriendo compensación en la parte `lui`.

11.3.3 Solución

El compilador genera la secuencia correcta con compensación:

```

1      # Salida de caracter en t0 al TTY de Logisim
2      li    t1, 0xFF000004      # Direcci n TTY
3      sw    t0, 0(t1)          # Escribir car cter (ASCII)
```

Listing 11.2: Acceso a TTY Logisim con compensación de signo

El ensamblador `compile_to_logisim.py` expande `li t1, 0xFF000004` a la secuencia correcta de `lui + addi` con la compensación de bit de signo apropiada.

11.4 Complicación 3: Modelo de Memoria Estático y Funciones Recursivas

11.4.1 Descripción

El compilador TPP asigna **direcciones de memoria estáticas** a todas las variables en tiempo de compilación. Esto significa que si una función se llama a sí misma recursivamente, las variables de la segunda invocación sobrescriben las de la primera (comparten el mismo offset en sp).

11.4.2 Manifestación del Error

El siguiente programa en TPP produce resultados incorrectos si se trata de ejecutarlo recursivamente:

```

1 fun factorial(entero n) -> entero {
2     si (n == 0) {
3         retornar 1;
4     }
5     // ERROR: la llamada recursiva sobrescribe el valor de 'n'
6     retornar n * factorial(n - 1);
7 }

```

Listing 11.3: Intento de recursión en TPP (resultado incorrecto)

11.4.3 Causa Técnica

En un compilador con frames dinámicos de pila, cada invocación de función ajustaría el stack pointer (sp) para crear un nuevo frame propio. En TPP, sp permanece fijo y todos los offsets son absolutos desde el mismo sp base.

11.4.4 Solución Actual

Para la versión actual del compilador, la recursión se marcó como **no soportada** y se documenta como limitación conocida. Como trabajo futuro, se propone implementar un modelo de frame pointer (fp) dinámico similar al de la convención RISC-V completa.

11.5 Complicación 4: Formato del Archivo Hexadecimal para Logisim

11.5.1 Descripción

Logisim Evolution requiere un formato de archivo específico para cargar en su componente RAM: el encabezado v2.0 raw seguido de los valores hexadecimales de 32 bits. Este formato no es estándar ni el producido directamente por riscv64-unknown-elf-objcopy.

11.5.2 Problema Inicial

Al principio se intentó usar el toolchain GNU (`riscv64-unknown-elf-gcc + objcopy -O ihex`), pero el formato Intel HEX no es directamente interpretado por la versión del componente RAM de Logisim Evolution utilizada en el laboratorio.

11.5.3 Solución

Se adaptó el script `compile_to_logisim.py` para producir exactamente el formato requerido. El script lee el `output.s`, lo ensambla internamente instrucción por instrucción, y produce el archivo `output.hex` con el encabezado correcto.

11.6 Complicación 5: Conflictos Léxicos con Babel en LaTeX

11.6.1 Descripción

Durante la generación de esta documentación, el paquete `babel` con la opción `spanish` activaba caracteres activos (como `<` y `>`) que interferían con `TikZ` y con el paquete `listings` al mostrar código ensamblador con operadores de comparación.

11.6.2 Solución

Se utilizaron las opciones `es-noquoting` y `es-noshorthands` de `babel` para desactivar los atajos de caracteres específicos del español que conflictuaban con `TikZ`:

```
\usepackage[spanish,es-noquoting,es-noshorthands]{babel}
```

Listing 11.4: Configuración Babel para compatibilidad con `TikZ`

11.7 Complicación 6: Caracteres Especiales en Cadenas de Texto

11.7.1 Descripción

El lexer de TPP soporta cadenas de texto con secuencias de escape (`\n`, `\t`). Sin embargo, el generador de código debía emitir estas cadenas en la sección `.data` del ensamblador y el script de Python debía procesarlas para calcular correctamente el tamaño de la cadena en memoria.

11.7.2 Solución

Se implementó un pre-procesador de cadenas en `codegen.c` que convierte las secuencias de escape a sus equivalentes en código ASCII antes de emitirlas, y el script de Python trata cada carácter de la cadena (incluyendo el nulo terminal `\0`) como un byte independiente a emitir en memoria.

Capítulo 12 PRUEBAS Y DEMOSTRACIONES DEL COMPILADOR

12.1 Descripción de las Pruebas

Para validar el funcionamiento integral del compilador TPP se diseñaron programas de prueba que ejercitan cada fase del pipeline. Cada prueba muestra: el código fuente en TPP, el tipo de resultado esperado (correcto o error), y la salida producida por el compilador.

12.2 Prueba 1: Programa Correcto — Operaciones Básicas

12.2.1 Código Fuente

```
1 // Prueba básica: variables, operaciones y salida
2 entero a = 10;
3 entero b = 3;
4 entero resultado = a + b;
5 imprimir("Suma:");
6 imprimir(resultado);
7
8 resultado = a * b;
9 imprimir("Producto:");
10 imprimir(resultado);
```

Listing 12.1: Programa correcto: operaciones básicas

12.2.2 Salida Esperada del Compilador

El compilador procesa el programa sin errores, muestra la tabla de símbolos, el TAC y el ensamblador generado:

```
===== TABLA DE SIMBOLOS =====
Nombre      Categoria  Tipo    Ambito  Tam  Memoria
a           Variable   entero  0       1    0x00000000
b           Variable   entero  0       1    0x00000004
```



```

===== CODIGO INTERMEDIO (TAC) =====
t0 = 10
a = t0
t1 = 3
b = t1
t2 = a
t3 = b
t4 = t2 + t3
resultado = t4
print " Suma : "
t5 = resultado
print t5
t6 = a
t7 = b
t8 = t6 * t7
resultado = t8
print " Producto : "
t9 = resultado
print t9
=====

```

Figura 12.3: Ejecución del programa mostrando el código intermedio (TAC)

```

===== OPTIMIZACION DE CODIGO =====
No se encontraron optimizaciones aplicables.
[Programa / Nodo Raiz]
| [Lista Elementos / Instrucciones]
| | [Lista Elementos / Instrucciones]
| | | [Lista Elementos / Instrucciones]
| | | | [Lista Elementos / Instrucciones]
| | | | | [Lista Elementos / Instrucciones]
| | | | | | [Lista Elementos / Instrucciones]
| | | | | | | [Declaracion Var] Tipo: 'entero'
| | | | | | | | [Asignacion] a variable 'a'
| | | | | | | | | [Enter] 10
| | | | | | | [Declaracion Var] Tipo: 'entero'
| | | | | | | | [Asignacion] a variable 'b'
| | | | | | | | | [Enter] 3
| | | | | | | [Declaracion Var] Tipo: 'entero'
| | | | | | | | [Asignacion] a variable 'resultado'
| | | | | | | | | [Operacion] '+'
| | | | | | | | | | [Variable] a
| | | | | | | | | | [Variable] b
| | | | | | | [Imprimir]
| | | | | | | | [Cadena] " Suma : "
| | | | | | | [Imprimir]
| | | | | | | | [Variable] resultado
| | | | | | | [Asignacion] a variable 'resultado'
| | | | | | | | [Operacion] '*'
| | | | | | | | | [Variable] a
| | | | | | | | | [Variable] b
| | | | | | | [Imprimir]
| | | | | | | | [Cadena] " Producto : "
| | | | | | | [Imprimir]
| | | | | | | | [Variable] resultado
=====

```

Figura 12.4: Ejecución del programa mostrando las optimizaciones aplicadas

```

===== CODIGO FINAL (RV32I) =====
.section .text

# ---- Rutina: print_str (Escribe en TTY Logisim 0xff000004) ----
__print_str:
    li t1, 0xff000004
.print_loop:
    lb t0, 0(a0)
    beqz t0, .print_end
    add zero, zero, t0 # FIX: Estabilizar rs2 para Logisim TTY
    sw t0, 0(t1)
    addi a0, a0, 1
    j .print_loop
.print_end:
    ret

# ---- Rutina: soft_mul (Multiplicador de software) ----
__soft_mul:
    li t0, 0
    li t1, 0
.mul_loop:
    beq t1, a1, .mul_end
    add t0, t0, a0
    addi t1, t1, 1
    j .mul_loop
.mul_end:
    mv a0, t0
    ret

# ---- Rutina: print_num (Imprime un entero a ASCII en TTY) ----

```

Figura 12.5: Ejecución del programa mostrando el código final generado

12.3 Prueba 2: Error Léxico

12.3.1 Código Fuente con Error

```

1 // El gui n bajo no es v lido en TPP
2 entero mi_variable = 5; // ERROR: '_' no es v lido
3 imprimir(mi_variable);

```

Listing 12.2: Programa con error léxico: carácter inválido _

12.3.2 Salida del Compilador

Al encontrar el carácter `_`, el lexer reporta el error léxico con línea y columna exactas:

Error Léxico en la línea 2, columna 11: Carácter inválido `'_'`

Error Léxico en la línea 2, columna 20: Carácter inválido `'_'`

El compilador **no aborta**, sino que reporta todos los errores léxicos encontrados y continúa con los tokens válidos. Sin embargo, los identificadores afectados no serán reconocidos, lo que probablemente dispare errores sintácticos en cascada.

```

aron@MacBook-Air-de-Aron ~/githubRepo/tpp (compiler-B*) $ python3 compile_to_logisim.py test/prueba02.to
[*] Compilando 'test/prueba02.to' con el compilador TPP...

¡Ups! Hay un problema sintáctico en tu código.
Linea 2, Columna 10
Detalle: syntax error cerca del elemento 'i'

Error Léxico en la línea 2, columna 12: Carácter inválido '_'
=> [Panic Mode] Error sintáctico ignorado. Analizador recuperado en el ';'.

¡Ups! Hay un problema sintáctico en tu código.
Linea 3, Columna 14
Detalle: syntax error cerca del elemento 'i'

Error Léxico en la línea 3, columna 16: Carácter inválido '_'
=> [Panic Mode] Error sintáctico ignorado. Analizador recuperado en el ';'.

Análisis finalizado con 2 error(es) sintáctico(s).
[-] Error durante la compilación del código fuente TPP.
    
```

Figura 12.6: Captura de error léxico por carácter inválido

12.4 Prueba 3: Error Sintáctico

12.4.1 Código Fuente con Error

```

1 // Falta el parntesis de cierre en el si
2 entero x = 5;
3 si (x > 0 {           // ERROR: falta ')
4     imprimir("positivo");
5 }
    
```

Listing 12.3: Programa con error sintáctico: falta paréntesis

12.4.2 Salida del Compilador

Bison detecta el error sintáctico en la posición exacta. La recuperación de errores (*panic mode*) con el token error permite que el parser continúe analizando el resto del archivo:

```

¡Ups! Hay un problema sintáctico en tu código.
Linea 3, Columna 10
Detalle: syntax error, unexpected '{', expecting ')'
=> [Panic Mode] Error en condicion del SI. Recuperando...
    
```

```

aron@MacBook-Air-de-Aron ~/githubRepo/tpp (compiler-B*) $ python3 compile_to_logisim.py test/prueba03.to
[*] Compilando 'test/prueba03.to' con el compilador TPP...

¡Ups! Hay un problema sintáctico en tu código.
Linea 3, Columna 12
Detalle: syntax error cerca del elemento '{'

[-] Error durante la compilación del código fuente TPP.
    
```

Figura 12.7: Captura de error sintáctico por paréntesis faltante

12.5 Prueba 4: Error Semántico

12.5.1 Código Fuente con Error

```
1 // Se intenta usar una variable que no fue declarada
2 entero x = 5;
3 entero resultado = x + y; // ERROR: 'y' no declarada
4 imprimir(resultado);
```

Listing 12.4: Programa con error semántico: variable no declarada

12.5.2 Salida del Compilador

El analizador semántico detecta el uso de un identificador que no existe en la tabla de símbolos:

```
Error Semantico: Uso de variable no declarada: 'y'
Error Semantico (linea aprox.): Uso de variable no declarada: 'y'
===== TABLA DE SIMBOLOS =====
x          Variable   entero  0      1      0x00000000
resultado  Variable   entero  0      1      0x00000004
Total: 2 simbolos | Memoria usada: 8 bytes
[COMPILACION ABORTADA: se encontraron errores semanticos]
```

```

aron@MacBook-Air-de-Aron ~/githubRepo/tpp (compiler-B*) $ python3 compile_to_logisim.py test/prueba04.to
[*] Compilando 'test/prueba04.to' con el compilador TPP...

Análisis sintáctico exitoso. El código es válido.

===== ÁRBOL SINTÁCTICO ABSTRACTO (AST) =====
[Programa / Nodo Raiz]
| [Lista Elementos / Instrucciones]
| | [Lista Elementos / Instrucciones]
| | | [Lista Elementos / Instrucciones]
| | | | [Declaracion Var] Tipo: 'entero'
| | | | | [Asignacion] a variable 'x'
| | | | | [Enter] 5
| | | | [Declaracion Var] Tipo: 'entero'
| | | | | [Asignacion] a variable 'resultado'
| | | | | | [Operacion] '+'
| | | | | | | [Variable] x
| | | | | | | [Variable] y
| | | [Imprimir]
| | [Variable] resultado
=====

===== ANALISIS SEMANTICO =====
Error semántico: Variable no declarada: 'y'
Error semántico: Variable no declarada: 'y'
Error semántico: Uso de variable no declarada: 'y'
===== TABLA DE SIMBOLOS =====
Nombre          Categoria  Tipo      Ambito  Tam      Memoria
-----
resultado       Variable  entero    0        1      0x00000004
x               Variable  entero    0        1      0x00000000
-----
Total: 2 simbolos | Memoria usada: 8 bytes (2 words)
=====
Análisis completado con 3 error(es) semantico(s).
[-] Error durante la compilación del código fuente TPP.

```

Figura 12.8: Captura de error semántico por variable no declarada

12.6 Prueba 5: Programa con Funciones

12.6.1 Código Fuente

```
1 fun duplicar(entero n) -> entero {
2     retornar n * 2;
3 }
4
5 fun main() {
6     entero valor = 7;
7     entero doble = duplicar(valor);
8     imprimir("Doble de 7:");
9     imprimir(doble);
10 }
```

Listing 12.5: Programa correcto con declaración y llamada de función

12.6.2 Salida Esperada

===== TABLA DE SIMBOLOS =====

Nombre	Categoria	Tipo	Ambito	Tam	Memoria
.ra_main	Variable	entero	0	1	0x00000008
main	Funcion	ninguno	0	-	0x00000000
.ra_duplicar	Variable	entero	0	1	0x00000000
duplicar	Funcion	entero	0	-	0x00000000

Total: 4 simbolos | Memoria usada: 20 bytes (5 words)

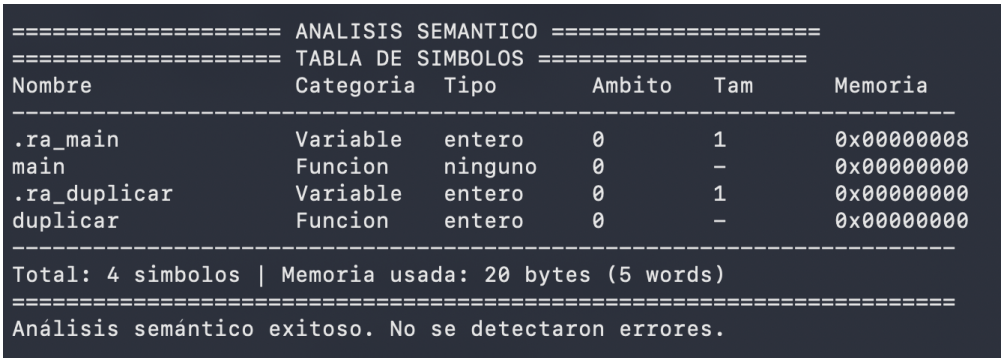


Figura 12.9: Ejecución del programa con funciones

12.7 Prueba 6: Optimización de Constantes

12.7.1 Código Fuente

```

1 entero a = 10 * 2 + 3;    // Debe plegarse a 23
2 entero b = a + 0;        // Simplificacion algebraica: b = a
3 entero c = 1 * a;        // Simplificacion algebraica: c = a
4 imprimir(a);
5 imprimir(b);
6 imprimir(c);

```

Listing 12.6: Programa que activa plegado de constantes y simplificación algebraica

12.7.2 Salida del Optimizador

===== OPTIMIZACION DE CODIGO =====

Optimizaciones aplicadas:

- Plegado de constantes: 2 (10*2=20, 20+3=23)
- Simplificacion algebraica: 2 (a+0->a, 1*a->a)
- Eliminacion de codigo muerto: 0

Total: 4 optimizaciones

```

===== OPTIMIZACION DE CODIGO =====
Optimizaciones aplicadas:
- Plegado de constantes: 2
- Simplificacion algebraica: 2
Total: 4 optimizaciones
[Programa / Nodo Raiz]
| [Lista Elementos / Instrucciones]
| | [Lista Elementos / Instrucciones]
| | | [Lista Elementos / Instrucciones]
| | | | [Lista Elementos / Instrucciones]
| | | | | [Lista Elementos / Instrucciones]
| | | | | [Declaracion Var] Tipo: 'entero'
| | | | | | [Asignacion] a variable 'a'
| | | | | | | [Enterol] 23
| | | | | [Declaracion Var] Tipo: 'entero'
| | | | | | [Asignacion] a variable 'b'
| | | | | | | [Variable] a
| | | | | [Declaracion Var] Tipo: 'entero'
| | | | | | [Asignacion] a variable 'c'
| | | | | | | [Variable] a
| | | | | [Imprimir]
| | | | | | [Variable] a
| | | | | [Imprimir]
| | | | | | [Variable] b
| | | | | [Imprimir]
| | | | | | [Variable] c
=====

```

Figura 12.10: Ejecución mostrando optimizaciones aplicadas

12.8 Prueba 7: Programa con Bucles y Arreglos

12.8.1 Código Fuente

```

1 entero arr[5];
2 entero i;
3 para (entero k = 0; k < 5; k = k + 1) {
4     arr[k] = k * 2;
5 }
6 para (entero j = 0; j < 5; j = j + 1) {
7     imprimir(arr[j]);
8 }

```

Listing 12.7: Programa con arreglo y bucle para

12.8.2 Salida Esperada

El programa imprime: 0, 2, 4, 6, 8 (los primeros 5 múltiplos de 2).

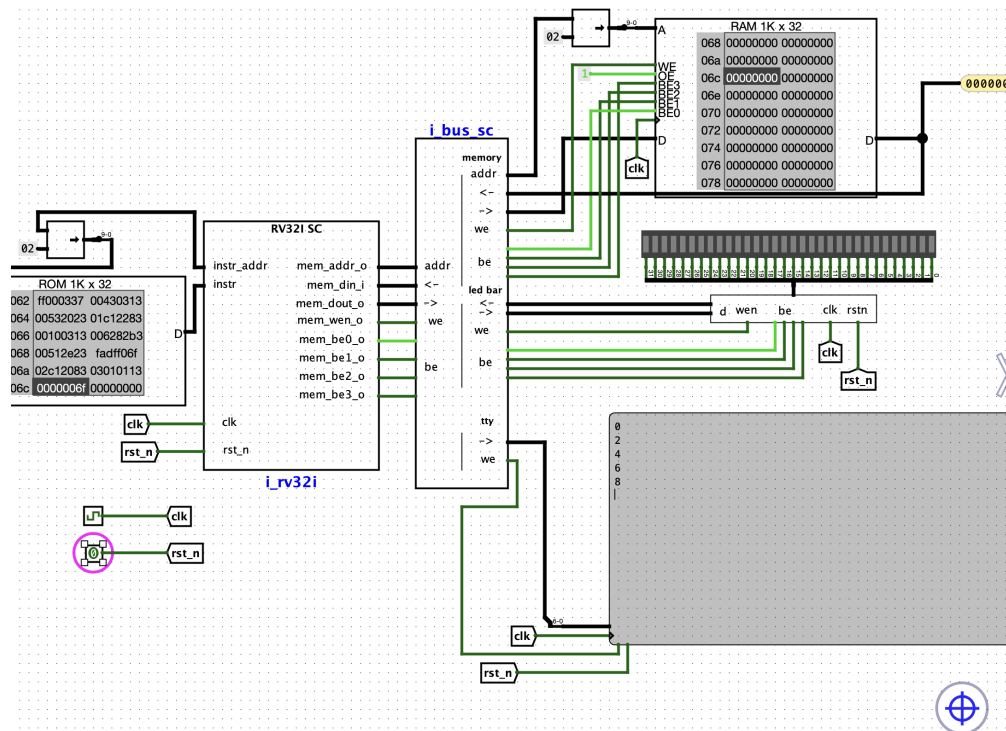


Figura 12.11: Ejecución del programa con arreglos y bucles para

12.9 Prueba 8: Prueba Integral — Backend

12.9.1 Código Fuente

```
1 fun sumarArreglo(entero n) -> entero {
2     entero suma = 0;
3     entero arr[5];
4     arr[0] = 1;
5     arr[1] = 2;
6     arr[2] = 3;
7     arr[3] = 4;
8     arr[4] = 5;
9     para (entero i = 0; i < n; i = i + 1) {
10         suma = suma + arr[i];
11     }
12     retornar suma;
13 }
14
15 fun main() {
16     entero total = sumarArreglo(5);
17     imprimir("Suma 1..5:");
18     imprimir(total);
19 }
```

Listing 12.8: Prueba backend: sumar elementos de arreglo

12.9.2 Resultado en Logisim

Con el archivo `output.hex` cargado en la RAM del procesador Logisim, la ejecución imprime 15 por el TTY del circuito.

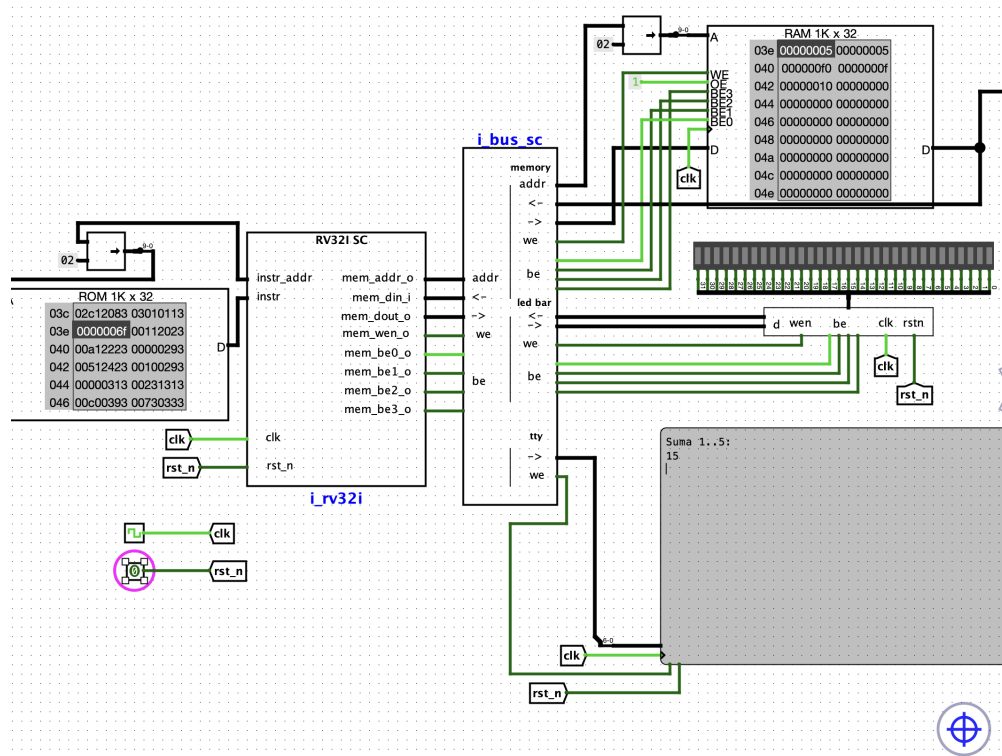


Figura 12.12: Ejecución del programa en Logisim Evolution — salida TTY

Capítulo 13 MANUAL DE USUARIO

13.1 Sintaxis del Lenguaje TPP

El lenguaje TPP utiliza palabras reservadas en español y sigue una estructura similar a C o Java. Todo archivo fuente debe tener la extensión .to. A continuación se detalla cómo utilizar cada una de las estructuras del lenguaje.

13.1.1 Tipos de Datos y Variables

TPP soporta variables entero y decimal, así como arreglos estáticos. Todas las sentencias deben terminar en punto y coma (;).

```
1 // Declaracion e inicializacion
2 entero edad = 20;
3 decimal pi = 3.14;
4
5 // Declaracion multiple
6 entero x, y, z;
7
8 // Declaracion de arreglos (el tamaño debe ser un número
   literal)
9 entero notas[5];
10 notas[0] = 15;
11 notas[1] = 18;
```

Listing 13.1: Declaración de variables y arreglos

13.1.2 Entrada y Salida

```
1 // Imprimir cadenas de texto o variables
2 imprimir("Bienvenido al sistema");
3 imprimir(edad);
4
5 // Leer un valor desde consola hacia una variable
6 leer(edad);
```

Listing 13.2: Operaciones de Entrada y Salida

13.1.3 Expresiones y Operadores

El lenguaje TPP permite construir expresiones aritméticas, lógicas y relacionales complejas respetando la precedencia matemática estándar (de mayor a menor prioridad):

- **Agrupación:** Paréntesis () para forzar la prioridad de evaluación.
- **Aritméticos:** Multiplicación *, División / (prioridad alta). Suma +, Resta - (prioridad baja).
- **Relacionales:** Mayor >, Menor <, Mayor o Igual >=, Menor o Igual <=.
- **Igualdad:** Igual que ==, Diferente de !=.

Tipos en Expresiones

TPP realiza **conversiones implícitas (coerción)** de tipos cuando se mezclan enteros y decimales en una operación aritmética. El resultado automáticamente se promoverá a decimal para no perder precisión. Sin embargo, los arreglos no pueden ser operados directamente (solo sus elementos individuales).

13.1.4 Estructuras Condicionales: si / sino

La estructura condicional evalúa una expresión. Si es diferente de cero (verdadero), ejecuta el primer bloque; de lo contrario, el bloque sino.

```

1 si (edad >= 18) {
2     imprimir("Es mayor de edad");
3 } sino {
4     imprimir("Es menor de edad");
5 }
```

Listing 13.3: Uso de si y sino

13.1.5 Selección Múltiple: depende / caso

Equivalente a la construcción switch/case de lenguajes de la familia C. Permite evaluar el valor de una variable y ejecutar exclusivamente el bloque que coincida con el caso correspondiente.

Detalles importantes:

- A diferencia de C, **no es necesario utilizar un equivalente a break**. La semántica de TPP asume que, una vez que un caso se ejecuta, el flujo del programa sale automáticamente del bloque depende.

- Los casos solo pueden comparar contra literales constantes, no contra otras variables.
- El bloque otros actúa de forma idéntica a default, ejecutándose si ninguna de las condiciones anteriores se cumplió. Su presencia es opcional.

```

1 entero opcion = 2;
2 depende (opcion) {
3     caso 1: {
4         imprimir("Opcion 1 seleccionada");
5     }
6     caso 2: {
7         imprimir("Opcion 2 seleccionada");
8     }
9     otros: {
10        imprimir("Opcion no valida");
11    }
12 }

```

Listing 13.4: Uso de depende y caso

13.1.6 Bucles: mientras y para

Los bucles permiten repetir un bloque de código. TPP soporta dos construcciones iterativas principales.

1. Bucle mientras (while): Se evalúa la condición de continuación antes de cada iteración. Si es verdadera, se ejecuta el cuerpo del bucle. Útil cuando no se conoce de antemano el número de iteraciones.

2. Bucle para (for): Es un bloque compacto que concentra la lógica de iteración. Requiere exactamente tres partes separadas por punto y coma:

1. **Inicialización:** Se ejecuta una sola vez al entrar al bucle. Puede declarar una nueva variable iteradora (ej. entero i = 0) o inicializar una existente.
2. **Condición:** Expresión booleana evaluada antes de cada iteración.
3. **Actualización:** Instrucción que se ejecuta al final de cada iteración, típicamente para incrementar el contador.

```

1 // Bucle MIENTRAS
2 entero contador = 0;
3 mientras (contador < 5) {
4     imprimir(contador);
5     contador = contador + 1; // No existe ++ en TPP
6 }
7
8 // Bucle PARA (inicializacion; condicion; incremento)
9 para (entero i = 0; i < 10; i = i + 1) {

```



```
10     imprimir(i);  
11 }
```

Listing 13.5: Uso de bucles

13.1.7 Funciones: fun

Las funciones se declaran con la palabra `fun`. Pueden tener parámetros y opcionalmente devolver un valor con la flecha `->`. La palabra `retornar` devuelve el resultado.

```
1 // Funcion que retorna un valor  
2 fun sumar(entero a, entero b) -> entero {  
3     retornar a + b;  
4 }  
5  
6 // Funcion sin valor de retorno (procedimiento)  
7 fun saludar(entero veces) {  
8     para (entero i = 0; i < veces; i = i + 1) {  
9         imprimir("Hola");  
10    }  
11 }  
12  
13 // Punto de entrada principal (opcional, por convencion)  
14 fun main() {  
15     entero res = sumar(5, 3);  
16     imprimir(res);  
17 }
```

Listing 13.6: Definicion de funciones

13.2 Guía de Compilación y Ejecución en Logisim

Para llevar un programa fuente `.to` hasta el procesador simulado en Logisim, siga estos pasos:

13.2.1 1. Compilación del código fuente

Ejecute el compilador pasándole el archivo `.to`. Si es exitoso, generará el archivo ensamblador `output.s`.

```
1 $ ./compilador programa.to
```

Si hay errores léxicos, sintácticos o semánticos, el compilador los reportará en la consola y abortará el proceso.

13.2.2 2. Generación del Código Hexadecimal

El ensamblador de Logisim requiere que las instrucciones estén en formato hexadecimal (v2.0 raw). Se utiliza el script de Python incluido en el repositorio:

```
$ python3 compile_to_logisim.py programa.to
```

El script leerá `output.s` y generará archivos hexadecimales (por defecto produce `output.hex`, `rom.hex` y `ram.hex` si la arquitectura del circuito separa las memorias de instrucciones y datos).

13.2.3 3. Carga en Logisim Evolution

1. Abra su proyecto de procesador RV32I en **Logisim Evolution**.
2. Localice el componente de memoria **ROM** (memoria de instrucciones) y/o **RAM** (memoria de datos).
3. Haga **click derecho** sobre el componente de memoria.
4. Seleccione **Load Image...** (Cargar Imagen).
5. Seleccione el archivo generado (`rom.hex`, `ram.hex` u `output.hex` según correspon-da).
6. Inicie el reloj (Ctrl+K o menú Simulate → Auto-Tick) para ver el programa ejecutarse.
7. Si el programa utiliza la instrucción imprimir, abra el componente TTY para ver la salida del programa.

Capítulo 14 Conclusiones y Trabajo Futuro

14.1 Resumen del Proceso de Compilación

El compilador TPP implementa el pipeline completo de traducción desde un lenguaje de programación imperativo en español hasta instrucciones ejecutables en hardware real. A continuación se resume el recorrido de un programa a través de cada fase:

Fase	Archivo	Transformación
Léxico	<code>lexer.l</code>	Texto → tokens (NUM_ENTERO, ID, SI, ...)
Sintáctico	<code>parser.y</code>	Tokens → AST jerárquico
Semántico	<code>semantic.c</code>	AST → AST anotado con tipos y offsets de memoria
Tabla de símb.	<code>syntab.c</code>	Registro de variables/funciones con sus direcciones
Optimización	<code>opt.c</code>	AST → AST simplificado (plegado, algebraica, muerto)
IR/TAC	<code>ir.c</code>	AST → representación plana de 3 direcciones (diagnóstico)
Codegen	<code>codegen.c</code>	AST → instrucciones RV32I en <code>output.s</code>

Cuadro 14.1: Resumen del pipeline completo

14.2 Logros Implementados

14.2.1 Frontend

- **Analizador léxico** completo con 30+ tokens, soporte de comentarios, cadenas con secuencias de escape, y reporte preciso de errores con línea y columna.

- **Analizador sintáctico** LALR(1) con gramática completa, resolución de ambigüedades (precedencia de operadores, dangling else), y recuperación de errores en modo pánico.
- **Analizador semántico** con verificación de tipos, detección de variables no declaradas, redefinición de símbolos y anotación de direcciones de memoria en el AST.
- **Tabla de símbolos** con scoping anidado, asignación estática de memoria y visualización detallada al final del análisis.

14.2.2 Backend

- **Optimizador de AST** con 4 pases: plegado de constantes, simplificación algebraica, eliminación de código muerto y propagación de constantes.
- **Generador de código intermedio** (TAC) para diagnóstico y comprensión del proceso.
- **Generador de código RV32I** completo con:
 - Generación bifásica para soporte de funciones.
 - Convención de llamada RISC-V (a0–a7, ra).
 - Prólogo y epílogo completo de funciones (salvar/restaurar ra).
 - Acceso a arreglos con cálculo dinámico de dirección.
 - Multiplicación por software (`__soft_mul`).
 - Impresión de enteros como ASCII (`__print_num`).
 - Salida TTY a la dirección Logisim `0xff000004`.

14.3 Construcciones Soportadas

Característica	Estado
Variables enteras y decimales	✓ Implementado
Arreglos estáticos	✓ Implementado
Operadores aritméticos (+, -, *, /)	✓ Implementado
Operadores relacionales (==, !=, <, >, <=, >=)	✓ Implementado
Estructura condicional si/sino	✓ Implementado
Bucle mientras	✓ Implementado
Bucle para	✓ Implementado
Estructura depende/caso	✓ Implementado
Declaración y llamada de funciones	✓ Implementado
Parámetros por valor	✓ Implementado
Variables globales	✓ Implementado
Imprimir cadenas y enteros	✓ Implementado
Comentarios de línea (//)	✓ Implementado
Optimización de constantes	✓ Implementado
Recursión	~ Parcial (requiere stack dinámico)
Tipo decimal en codegen	× Pendiente
Paso de arreglos a funciones	× Pendiente

Cuadro 14.2: Estado de las características del compilador

14.4 Limitaciones Actuales

14.4.1 Modelo de Memoria Estático

El compilador TPP utiliza un modelo de memoria **completamente estático**: todos los offsets se calculan en tiempo de compilación y son fijos. Esto tiene las siguientes implicaciones:

- **Recursión limitada:** Una función recursiva sobrescribiría sus propias variables locales al ser llamada de nuevo, ya que todas las invocaciones comparten el mismo espacio de memoria.
- **Sin allocación dinámica:** No existe instrucción malloc equivalente.
- **Arreglos de tamaño fijo:** Los arreglos deben declararse con tamaño constante conocido en compilación.

14.4.2 Sin guión bajo en identificadores

El lexer solo reconoce `[a-zA-Z][a-zA-Z0-9]*` como identificadores. El carácter `_` produce un error léxico. Se recomienda camelCase.

14.5 Trabajo Futuro

Las siguientes mejoras podrían extender las capacidades del compilador:

1. **Stack dinámico por función:** Implementar un frame pointer (fp) separado del stack pointer para que cada invocación de función tenga su propio frame. Esto habilitaría la recursión completa.
2. **Soporte de tipo decimal en codegen:** Generar instrucciones de punto flotante o emular operaciones decimales con enteros de punto fijo.
3. **Paso de arreglos por referencia:** Pasar la dirección base del arreglo como puntero al argumento `a0`, permitiendo que las funciones modifiquen arreglos externos.
4. **Múltiples archivos fuente:** Soporte para importar otros archivos `.to` y compilarlos juntos.
5. **Optimizaciones adicionales:** Eliminación de subexpresiones comunes, renombramiento de registros, inlining de funciones pequeñas.
6. **Soporte de guión bajo:** Extender el lexer para permitir `_` en identificadores (`[a-zA-Z_][a-zA-Z0-9_*`
7. **Extensión de ISA:** Soporte para la extensión M de RISC-V (multiplicación nativa con `mul`, `div`) una vez que el procesador Logisim la implemente.

14.6 Ejemplo Completo: De Fuente a Binario

El siguiente ejemplo muestra el recorrido completo de una línea de código:

```
1 entero resultado = 10 * 2 + 3;
2 imprimir(resultado);
```

Listing 14.1: Código fuente TPP

Tokens producidos por el lexer:

```
TIPO_ENTERO ID("resultado") ASIG NUM_ENTERO(10) MULT
NUM_ENTERO(2) MAS NUM_ENTERO(3) PUNTOCOMA
IMPRIMIR PARI ID("resultado") PARD PUNTOCOMA
```

AST (antes de optimización):

```

[Declaracion Var] entero
  [Asignacion] resultado
    [Operacion] '+'
    [Operacion] '*'
    [Entero] 10
    [Entero] 2
    [Entero] 3
[Imprimir]
  [Variable] resultado

```

AST (después de optimización — plegado en cascada):

```

[Declaracion Var] entero
  [Asignacion] resultado
    [Entero] 23      ← 10*2=20, 20+3=23
[Imprimir]
  [Variable] resultado

```

Código RV32I generado:

```

1  li    t0, 23          # Constante plegada (era 10*2+3)
2  sw    t0, 0(sp)       # resultado = 23
3  lw    t0, 0(sp)       # Cargar resultado
4  mv    a0, t0
5  call  __print_num     # Imprimir el entero
6  li    t0, 10          # Salto de linea
7  sw    t0, 0xff000004

```

Listing 14.2: Ensamblador RV32I generado

Sin la optimización de plegado, se habrían generado 12 instrucciones adicionales para la multiplicación y la suma. Con la optimización, se reduce a 2 instrucciones esenciales.